

The Evolution of Large Language Model Architectures

A graduate-level monograph on what each generation of language models solved, what new constraint it introduced, and what it teaches us about inventing the next one.

Preface

Most popular accounts of large language models read like a parade of names: n -grams, then RNNs, then Transformers, then ChatGPT. This is not history; it is a list. The real story is more disciplined and more interesting. Each architecture in the lineage of language modeling exists because it **reallocated a constraint**. It traded a problem the previous architecture could not solve for a different problem we were better equipped to handle.

This monograph treats architectural evolution as a sequence of *constraint reallocations*. Every successful model class — n -grams, neural probabilistic LMs, recurrent networks, gated recurrence, attention, Transformers, mixture-of-experts, state-space models — is studied through four questions:

1. **What constraint did it relax?** What was the previous bottleneck (sparsity, vanishing gradients, sequential compute, $\mathcal{O}(N^2)$ memory)?
2. **What new constraint did it introduce?** What did the new design make harder or more expensive?
3. **Why did it work empirically?** What property of language, optimization, or hardware did it exploit?
4. **Why did it eventually plateau?** What did scale, data, or hardware reveal that was previously invisible?

The reason this matters is methodological. The history of LLMs has been steered as much by hardware curves and optimizer behavior as by ideas about language. **Architecture is the surface where statistics, optimization, and systems collide**. Looking backward through the constraint-reallocation lens, you can see that:

- The dominance of distributed representations was less about “semantics” than about smoothing in high dimensions.
- Recurrence was conceptually correct but numerically wrong; gating was a numerical fix to a numerical problem.
- Attention won not because it had a better inductive bias for language, but because its **shorter gradient paths** and **denser parallelism** matched accelerator economics.
- The Transformer’s enduring power lies less in self-attention and more in its **residual stream** as a writable, layer-wise communication bus.
- Modern advances (RMSNorm, SwiGLU, RoPE, GQA, FlashAttention, MoE) are not “tricks” but careful redistributions of compute, memory bandwidth, and gradient flow.

The thesis of the book: **future architectures will be invented by people who can read a constraint table and notice which row is suspiciously expensive**. Not by people who try to “make the Transformer smarter.” Architecture matters more than tricks because tricks live and die with one model family, while a good architecture re-houses many tricks at once.

A note on tone. The chapters mix intuition and formalism deliberately. Where a paragraph of prose can replace an equation, it does. Where an equation clarifies *why* a thing works (rather than merely *what* it computes), it stays. We are not writing for marketers; we are writing for the practitioner who will eventually have to fix something at 3 a.m. or invent the next thing at scale.

A note on omissions. This is not an exhaustive survey. Hundreds of important models — from CharRNNs to ELMo, from Reformer to RWKV — appear in passing or not at all. They do not all belong in a book about *evolution* unless they teach a lesson the lineage preserved.

Let us begin where every honest book about language models has to begin: with what a language model actually is.

Chapter 1. Foundations

1.1 What is a language model?

A language model is a probability distribution over sequences of discrete symbols. Given a vocabulary $\mathcal{V}$ and a sequence $x_{1:T} = (x_1, x_2, \dots, x_T)$ with each $x_t \in \mathcal{V}$, a language model assigns a number

$$p_\theta(x_{1:T}) \in [0, 1], \quad \sum_{x_{1:T}} p_\theta(x_{1:T}) = 1.$$

The variables x_t are tokens — bytes, characters, sub-words (BPE, WordPiece, SentencePiece), or full words. The model is parameterized by θ . We never store the joint distribution explicitly; we store a parameterization that lets us evaluate it.

Two equivalent factorizations dominate:

- **Causal / autoregressive (left-to-right):**

$$p_\theta(x_{1:T}) = \prod_{t=1}^T p_\theta(x_t \mid x_{<t}).$$

- **Masked / non-causal:** model $p_\theta(x_t \mid x_{\setminus t})$ for some masked position t , used at training time but giving up a clean joint at inference.

The autoregressive factorization is *exact* — it is just the chain rule of probability — but it is also a strong commitment. It says we will pay the cost of modeling each conditional. If we choose a wrong family for those conditionals, no amount of clever decoding repairs it.

1.2 Cross-entropy as a training objective

Given a true distribution q over sequences (the data) and a model p_θ , the cross-entropy

$$H(q, p_\theta) = -\mathbb{E}_{x \sim q}[\log p_\theta(x)]$$

is a measure of how many nats per token (bits per token in $\log_2$) the model spends representing the data. Minimizing $H(q, p_\theta)$ over θ is equivalent to maximum-likelihood estimation:

$$\hat{\theta} = \arg \min_{\theta} -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x^{(i)}).$$

The KL form makes the relationship to the data distribution explicit:

$$H(q, p_{\theta}) = H(q) + \text{KL}(q \parallel p_{\theta}).$$

$H(q)$ is the irreducible entropy of language — the part *no* model can compress away. The training objective is therefore the *excess* description length, $\text{KL}(q \parallel p_{\theta})$. This single equation already foreshadows the scaling laws of Chapter 13: there is a floor; models close in on it asymptotically; once close, returns diminish.

Two consequences are worth internalizing:

1. **Cross-entropy is a proper scoring rule.** A model that hedges loses; a model that is confidently wrong loses more; the unique minimizer is $p_{\theta} = q$.
2. **Negative log-likelihood is a description length.** Training a language model is, literally, learning to compress text. The connection to information theory in Chapter 10 is not metaphorical.

1.3 Why sequence modeling is hard

Three structural difficulties explain why language modeling resisted easy progress for decades.

Combinatorial explosion. The number of length- T sequences over $|\mathcal{V}|$ grows as $|\mathcal{V}|^T$. For $|\mathcal{V}| = 50,000$ and $T = 1,024$, this number dwarfs the number of atoms in the observable universe by hundreds of orders of magnitude. *Every nontrivial sequence we ever care about is statistically a singleton.* Generalization is mandatory; memorization is mathematically impossible.

Long-range dependence. Language is *not* Markovian of order k for any small k . A pronoun's referent can sit hundreds of tokens earlier; an open quotation, code block, or chess board can require global state. Local statistics work surprisingly far, but never far enough.

Heavy-tailed token distributions. Token frequency follows a power law (Zipf). The probability mass is dominated by a tiny “head,” but the *informativeness* per token is concentrated in a long tail (rare nouns, named entities, specialized terms). Any model that reduces the tail to noise will fail at the tasks people actually care about.

These three together imply: we need a model class that **shares parameters across sequences that differ in their surface form but share underlying structure, propagates information across long distances, and avoids a sparsity catastrophe in the tail**. The rest of the book is a chronicle of how each architecture attacks one or more of these and pays for the others.

1.4 Perplexity, bits-per-byte, and what loss numbers mean

A loss of 2.5 nats per token sounds abstract. Three derived metrics make it concrete.

Perplexity is $\text{ppl} = \exp(H(p, q))$. A perplexity of 12 means the model is, on average, “as uncertain” as a uniform distribution over 12 equally-likely options. Going from $\text{ppl} = 50$ to $\text{ppl} = 25$ doubles the precision of the predictive distribution, even though the loss only fell by $\ln 2 \approx 0.69$ nats. Perplexity is exponential in the loss, and the field’s intuition for “good loss” is much sharper in perplexity than in nats.

Bits per byte (BPB) or **bits per character (BPC)** normalizes for tokenizer differences. A model with vocabulary $|\mathcal{V}| = 32\text{k}$ may use fewer tokens per byte of text than one with $|\mathcal{V}| = 100\text{k}$; comparing their per-token cross-entropies is unfair. BPB renormalizes by the average bytes per token in the corpus:

$$\text{BPB} = \frac{H(p, q) \cdot \log_2 e}{\bar{b}_{\text{per token}}}.$$

Frontier LLMs achieve $\text{BPB} \approx 0.6$ on diverse text — meaning each byte requires about 0.6 bits of code on average, a compression ratio of more than $13\times$ over uncompressed UTF-8. This is the most architecture-agnostic loss metric and is used for cross-paper comparison.

Calibrated probabilities. A well-trained model’s predictive distributions should match their empirical correctness: when the model assigns 0.8 probability to a token, that token should occur 80% of the time. Modern instruction-tuned LLMs are *miscalibrated* (typically overconfident) due to the reward model’s sharpening pressure during RLHF. This matters for downstream tasks that rely on the probability surface (rejection sampling, uncertainty estimation, reasoning consistency).

Each metric stresses a different property: perplexity emphasizes mean predictability, BPB enables cross-tokenizer comparison, calibration tracks honesty.

1.5 Sampling: temperature, top- k , top- p

Once a model has been trained to predict a distribution, *how* you sample from it determines what the user experiences. Several knobs:

- **Greedy decoding.** Take $\arg \max p$ at each step. Deterministic, often dull, prone to repetition loops because once a high-probability suffix appears, it self-reinforces.
- **Temperature τ .** Sample from $\text{softmax}(z/\tau)$. $\tau = 1$ is the model’s distribution; $\tau \rightarrow 0$ approaches greedy; $\tau > 1$ flattens. Temperatures around 0.6–0.8 are typical for chat; near 0 for code completion or math.
- **Top- k sampling.** Keep only the k most probable tokens at each step; renormalize and sample. Cuts off the long tail entirely. Robust against pathological tokens but somewhat arbitrary.
- **Top- p (nucleus) sampling** (Holtzman et al. 2019). Keep the smallest set of tokens whose cumulative probability exceeds p (e.g., $p = 0.9$). Adapts the cutoff to the distribution’s shape: when the model is confident, the nucleus is small; when uncertain, the nucleus is wide.
- **Min- p sampling.** Keep tokens with probability $\geq p \cdot p_{\max}$. A scale-invariant nucleus filter; popular in recent open-source.
- **Repetition / presence penalties.** Down-weight tokens already emitted; rough heuristics that fight degeneration loops.

Sampling is part of the architecture’s *contract*: it is what converts a probabilistic predictor into a generator. Architectural changes that improve loss but produce harder-to-sample distributions (e.g., very peaky outputs) can degrade user-visible quality even when benchmark numbers improve.

1.6 The probabilistic frame as the right one

Why insist on probability? Because it makes the next chapters cohere. Cross-entropy gives a learning signal; perplexity gives a quality measure; sampling gives a generator; calibration gives an uncertainty estimate; KL divergence gives a distance between models for distillation and RLHF. The same probabilistic object underwrites pretraining, fine-tuning, RL with verifiable rewards, and inference.

Architectures that abandon the probabilistic frame (e.g., energy-based models, score-based models, diffusion LMs) have to rederive each of these notions in their own framework — and the LLM literature has, after several attempts, kept returning to autoregressive softmax-over-tokens as the simplest sufficient choice. The chapters that follow will not always be probabilistic in their primary motivation, but the probabilistic frame remains the connective tissue.

Chapter 2. Statistical Language Models

2.1 The Markov assumption and n -grams

Before neural networks, the standard answer to “how do we estimate $p(x_t \mid x_{<t})$ ” was to make it tractable by truncation:

$$p(x_t \mid x_{<t}) \approx p(x_t \mid x_{t-n+1}, \dots, x_{t-1}).$$

This is the **n -gram assumption**. With $n = 2$ (bigrams) the model uses one previous token; with $n = 5$ it uses four. Under this assumption, the maximum-likelihood estimator is just counting:

$$\hat{p}(x_t \mid x_{t-n+1:t-1}) = \frac{C(x_{t-n+1:t})}{C(x_{t-n+1:t-1})}.$$

Counts are stored in giant hash tables; inference is a lookup. For decades this was the dominant LM technology — in machine translation, ASR, and even early statistical chatbots.

2.2 The sparsity catastrophe

The n -gram model has one mortal flaw: most n -grams *that occur in test data have zero count in training data*. The probability of an unseen n -gram is then exactly zero, which makes the joint probability of any sentence containing it zero. This is unacceptable.

The response was a long literature on **smoothing** — ways to redistribute probability mass to unseen events. Two ideas dominated:

- **Backoff.** If $C(x_{t-2:t}) = 0$, fall back to $\hat{p}(x_t \mid x_{t-1})$. If that is zero, fall back to $\hat{p}(x_t)$. Katz backoff makes this rigorous with discounting.
- **Interpolation.** Always blend lower-order estimates: $\lambda_3 \hat{p}_3 + \lambda_2 \hat{p}_2 + \lambda_1 \hat{p}_1$, with $\sum \lambda = 1$, learned on held-out data.

The empirical winner was **Kneser–Ney smoothing**, whose central insight is sometimes called the “**continuation count**”:

The probability of a word in a backoff distribution should depend on **how many distinct contexts it appears in**, not how many times it appears overall.

Why? Consider the word *Francisco*. It appears often, but almost always after *San*. If we are backing off because the bigram (*reportedly, Francisco*) was unseen, $\hat{p}(\text{Francisco})$ should *not* be high, because *Francisco* essentially never appears as a free-standing token. Kneser–Ney’s continuation probability captures exactly this:

$$p_{\text{cont}}(w) \propto |\{v : C(v, w) > 0\}|.$$

The full modified Kneser–Ney formula combines absolute discounting with this continuation distribution at every order. It held the state-of-the-art in n -gram smoothing for a decade.

2.3 The hard ceiling

Even with the best smoothing, n -gram LMs hit three walls:

1. **Curse of dimensionality.** The number of distinct n -grams scales as $|\mathcal{V}|^n$. For $|\mathcal{V}| = 50,000$ and $n = 5$, that is 3×10^{23} cells — far more than any corpus could ever populate. Smoothing buys you a few extra orders of magnitude of *coverage*; it cannot give you the *generalization* you need.
2. **No similarity.** *Black sedan* and *dark car* are, to an n -gram, as different as *Black sedan* and *banana penguin*. There is no notion that two contexts could be similar. Every parameter is per-context.
3. **Truncated history.** Even at $n = 7$, the model cannot represent that the subject of the sentence two paragraphs ago controls verb agreement now.

The lesson the lineage preserved: **count-based, exact-match conditional probabilities are a dead end.** Some form of *parameter sharing across similar contexts* is mandatory.

This is exactly the constraint the next architecture relaxed.

2.4 Smoothing in detail: Laplace, Witten–Bell, Kneser–Ney

The textbook fix for zero-probability events is **Laplace smoothing** (add-one):

$$p(w \mid h) = \frac{c(h, w) + 1}{c(h) + |\mathcal{V}|}.$$

Robust against zeros, terrible at allocating probability mass — adding one to a 0 count is far too generous. Add- ϵ smoothing helps; better still is to *learn* how much to discount.

Witten–Bell smoothing allocates discounted mass proportional to the number of distinct contexts a word has appeared in. The intuition: a word that appears in many contexts is more likely to appear in a new one.

Kneser–Ney smoothing is the gold standard among classical methods. Two ideas:

1. **Absolute discounting.** Subtract a constant D from every observed count, redistributing the saved mass to a back-off distribution.

2. **Lower-order distribution by versatility.** Instead of using raw frequency for the back-off distribution, use *the number of distinct preceding words* a token appears with. A word like “Francisco” is frequent (because of “San Francisco”) but appears in few contexts; Kneser–Ney correctly downweights it as a back-off prediction.

Modified Kneser–Ney (Chen and Goodman, 1998) uses different discount values for counts of 1, 2, and ≥ 3 . Until 2003 (Bengio NPLM) and arguably until 2013 (Word2Vec), Kneser–Ney n -gram models were the strongest LMs ever built — and they still outperform small Transformers on tiny corpora.

2.5 Backoff and interpolation

When an n -gram is unseen, you have two combination strategies:

- **Backoff.** If $c(h, w) > 0$, use the n -gram estimate; otherwise, recursively use the $(n - 1)$ -gram estimate. The model has a discrete switch.
- **Interpolation.** Always use a weighted sum of all orders: $p(w | h) = \lambda_n p_n(w | h) + \lambda_{n-1} p_{n-1}(w | h_{-1}) + \dots$. Smoother in theory; a held-out set is required to estimate λ .

Modified Kneser–Ney can be expressed in either form; the interpolated version is generally preferred because it gives non-zero probability to all orders simultaneously, making perplexity smoother across rare contexts.

The deeper lesson — one that recurs throughout the neural era — is that *combining estimators of different specificity* is a generally good idea. RNN/LSTM ensembles, MoE mixtures, and even mixture-of-depth architectures are descendants of this exact instinct.

2.6 Cache and class-based language models

Two extensions that classical LMs explored before neural methods:

- **Cache models** (Kuhn & De Mori, 1990). Mix the static n -gram with a dynamic counter over the *recently observed* tokens. If “neural” appeared a few sentences ago, raise its probability for the next few sentences. A primitive form of adaptation — an architectural seed of what RNNs would later do continuously through their hidden state.
- **Class-based LMs** (Brown et al., 1992). Cluster tokens into classes $\mathcal{C}$, train an LM over class transitions, separately learn $p(w | \mathcal{C})$. Reduces parameter count and shares signal between similar tokens. The conceptual ancestor of **distributed representations** — the next chapter’s revolution.

The trajectory should be visible: every classical idea that smelled like architecture (sharing, smoothing across distance, dynamic context) gets *absorbed* into the neural era as a continuous, differentiable mechanism. The architectures of 2024 are sharper, more parallel, and trained at scale, but the constraints they relax were already legible to careful researchers in 1992.

Chapter 3. Neural Probabilistic Language Models (Bengio 2003)

3.1 The big idea

Yoshua Bengio and colleagues (NIPS 2000; JMLR 2003) proposed the simplest possible neural LM — and in doing so changed the field. Their architecture:

1. Map each word in the context window to a learned vector $\mathbf{e}(w) \in \mathbb{R}^d$.
2. Concatenate the context vectors $[\mathbf{e}(x_{t-n+1}); \dots; \mathbf{e}(x_{t-1})]$.
3. Pass through a tanh hidden layer.
4. Apply a softmax to predict x_t .

In equations:

$$\mathbf{h} = \tanh(W[\mathbf{e}(x_{t-n+1}); \dots; \mathbf{e}(x_{t-1})] + \mathbf{b}),$$

$$p(x_t = w \mid x_{<t}) \propto \exp(\mathbf{u}_w^\top \mathbf{h} + c_w).$$

Training jointly learns the embeddings $\mathbf{e}$, the hidden weights W , and the output projection.

3.2 Why this solved the sparsity problem

The crucial property is **distributed representations as parameter sharing**. If *cat* and *dog* end up close in embedding space, then any context the model has seen with *cat* contributes signal to predictions about *dog*. We have moved from $\mathcal{O}(|\mathcal{V}|^n)$ disjoint table cells to $\mathcal{O}(d|\mathcal{V}|)$ shared parameters that cooperate.

This is the same trick by which generalized linear models beat indicator-variable expansions: replace one-hot features with learned features that span a smooth space. In high dimensions, smoothness is the only generalizer that works.

A closely related view: the softmax over the hidden state is a **log-bilinear scoring function**, $\text{score}(w \mid \mathbf{h}) = \mathbf{u}_w^\top \mathbf{h}$. The energy landscape is shaped so that semantically/contextually plausible words get high scores. The cross-entropy loss is exactly the Boltzmann-style likelihood of this energy.

3.3 What it didn't fix

Two ceilings remained:

- **Fixed context.** The Bengio model still windowed history at $n - 1$ tokens. It generalized *across* contexts, not *across distance*. Long-range dependency was still untouched.
- **Compute.** The softmax over $|\mathcal{V}|$ was the dominant cost. Decades of clever workarounds followed (hierarchical softmax, NCE, sampled softmax) — important, but not architectural.

Lesson preserved by the lineage: **embed first, predict later**. Every modern LLM still does exactly this. Token embeddings are the universal first layer.

3.4 The architecture, fully drawn

The Bengio NPLM has four separable parts:

1. **An embedding lookup.** $C \in \mathbb{R}^{|\mathcal{V}| \times d}$, retrieved by integer index for each context word.
2. **A concatenation.** Window of $n - 1$ context words yields a feature vector of size $(n - 1) \cdot d$.
3. **A hidden layer.** $H \in \mathbb{R}^{(n-1)d \times d_h}$ followed by $\tanh$.
4. **An output layer.** $U \in \mathbb{R}^{d_h \times |\mathcal{V}|}$ producing logits, plus an optional skip connection from the input concatenation directly to the output.

Total parameters: $|\mathcal{V}|d + (n - 1)d \cdot d_h + d_h \cdot |\mathcal{V}|$. For the Brown corpus setup ($|\mathcal{V}| = 17,964$, $n = 6$, $d = 60$, $d_h = 100$), about 4×10^6 parameters. By 2003 standards this was non-trivial; by 2025 standards, it is the embedding table of a tiny LM.

The empirical result: this 4M-parameter neural model beat Kneser–Ney trigrams on the Brown and AP corpora by a perplexity factor of roughly 20%. On larger corpora the gap widened. The architectural skeleton of every modern LLM is here: *embedding lookup, dense feature processing, softmax over vocabulary, trained end-to-end with cross-entropy*. RNNs, LSTMs, Transformers — all elaborations on this skeleton.

3.5 The softmax bottleneck and its workarounds

The dominant cost of the Bengio NPLM was the softmax over $|\mathcal{V}|$ output classes. For each training example, the model had to:

- Compute $|\mathcal{V}|$ logits via the matmul $\mathbf{h}^\top U$.
- Exponentiate, sum, normalize.
- Backpropagate through the same.

For $|\mathcal{V}| = 100,000$ and $d_h = 1,000$, each forward pass spent $\sim 10^8$ FLOPs on the softmax alone — an order of magnitude more than the rest of the network. Three workarounds shaped the next decade:

- **Hierarchical softmax** (Morin & Bengio 2005). Arrange the vocabulary in a binary tree (often built by frequency clustering or Huffman code). Each prediction becomes a sequence of $\log_2 |\mathcal{V}|$ binary decisions along a path. Reduces output cost from $\mathcal{O}(|\mathcal{V}|)$ to $\mathcal{O}(\log |\mathcal{V}|)$ at the cost of distorting the training distribution slightly.
- **Noise Contrastive Estimation (NCE)** (Gutmann & Hyvärinen 2010; Mnih & Teh 2012). Instead of normalizing over $|\mathcal{V}|$, train a binary classifier to distinguish true samples from noise samples. The exact softmax becomes optional; the model learns *score functions* whose ratio approximates the true probability.
- **Sampled softmax / negative sampling** (Mikolov et al. 2013, with Word2Vec). At each step, only update the true target plus a handful of randomly sampled negatives. Conceptually a noisy NCE; in practice extremely fast and the workhorse of Word2Vec.

These are not architectural improvements in the structural sense; they are *training-objective modifications* that allow the same architecture to scale. The lesson, which recurs in modern MoE training and contrastive embedding training: when the sum-over-vocabulary is the bottleneck, find a Monte Carlo or hierarchical surrogate that trades exactness for tractability. The exact softmax has only become routine again because GPUs made it cheap; for a long stretch, every LM that scaled used some form of approximation here.

Chapter 4. Word Embeddings

4.1 The decoupling

If embeddings are the workhorse of generalization, perhaps we don't need a *full* language model to learn them. Around 2013, three families showed this was true:

- **Word2Vec** (Mikolov et al., 2013): two shallow architectures — *Skip-gram* (predict context from target) and *CBOW* (predict target from context). Trained with hierarchical softmax or negative sampling.
- **GloVe** (Pennington et al., 2014): factorize the *log* of the word-word co-occurrence matrix. Connects neural embeddings to classical matrix factorization.
- **fastText** (Bojanowski et al., 2016): represent each word as the sum of its character *n*-gram embeddings. Solves out-of-vocabulary in a clean way.

4.2 Distributional semantics, formally

The deep claim, due to Firth ("you shall know a word by the company it keeps") and made empirical by these models, is that **the local co-occurrence statistics of words approximate their meaning**. The mathematical content is that Skip-gram with negative sampling implicitly factorizes the **shifted pointwise mutual information matrix**:

$$\text{PMI}(w, c) - \log k = \mathbf{e}(w)^\top \mathbf{e}(c),$$

where k is the number of negative samples (Levy & Goldberg 2014). All three families — Word2Vec, GloVe, classical LSA — are factorizing variants of co-occurrence; they differ in the loss, weighting, and what they take logs of.

4.3 The geometry surprise

Trained word vectors revealed an unreasonable amount of structure:

- Linear directions encoded gender, tense, plurality.
- $\mathbf{e}(\text{king}) - \mathbf{e}(\text{man}) + \mathbf{e}(\text{woman}) \approx \mathbf{e}(\text{queen})$.
- Cosine similarity recovered semantic similarity.

The geometry exists because the loss is bilinear, the data has additive compositionality (gender and royalty co-vary roughly independently in the corpus), and the optimization is gentle enough to find a clean factorization. None of it was guaranteed in advance.

4.4 Why static embeddings were a dead end

A single vector $\mathbf{e}(\text{bank})$ cannot represent both *river bank* and *investment bank*. Polysemy is everywhere; it is the rule, not the exception. Even outside polysemy, *contextual* meaning matters — *bright* means different things modifying *student* vs. *star*. Static embeddings refuse to acknowledge this.

The fix was to make the embedding *a function of the context* — i.e., the embedding lives inside the LM, not before it. This is what RNNs and then Transformers did. Word embeddings remained as the *input layer*; everything above became contextual.

Lesson: **the right object is a contextual representation, not a token representation.**

4.5 Skip-gram vs CBOW: the math

Word2Vec offered two architectures, and the choice mattered.

Skip-gram predicts context from target. Given a target word w_t and a context word w_{t+j} within window $\pm c$:

$$p(w_{t+j} \mid w_t) = \frac{\exp(\mathbf{v}'_{w_{t+j}} \mathbf{v}_{w_t})}{\sum_{w \in \mathcal{V}} \exp(\mathbf{v}'_w \mathbf{v}_{w_t})}.$$

Two embedding matrices: $\mathbf{v}$ for “input” (the target) and $\mathbf{v}'$ for “output” (the context). The trained representation kept and used downstream is conventionally $\mathbf{v}$.

CBOW (Continuous Bag of Words) predicts target from context. Average the context embeddings, then predict the target:

$$p(w_t \mid w_{t-c:t+c}) = \text{softmax}_{w_t} \left(W \cdot \frac{1}{2c} \sum_{j \neq 0} \mathbf{v}_{w_{t+j}} \right).$$

The empirical difference: **Skip-gram is better at rare words** (each rare word gets many gradient updates, one per context appearance) while **CBOW trains faster** (one update per target token regardless of context). At scale, Skip-gram was the dominant choice.

4.6 Negative sampling, derived

The full softmax over $|\mathcal{V}|$ is too slow. Negative sampling replaces it with a logistic discrimination problem.

For Skip-gram, the original objective is $\log p(w_O \mid w_I)$. Negative sampling instead optimizes:

$$\log \sigma(\mathbf{v}'_{w_O} \mathbf{v}_{w_I}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-\mathbf{v}'_{w_i} \mathbf{v}_{w_I})],$$

with σ the sigmoid and $P_n(w) \propto U(w)^{3/4}$ a smoothed unigram distribution (the $3/4$ exponent is empirical; it slightly upweights rare words relative to raw unigram).

This is *not* the same objective as the full softmax — it does not normalize and is therefore not a probability — but its gradient pushes embeddings toward an equivalent geometry for similarity-based downstream use. With $k = 5\text{--}15$ negatives, training is roughly $k + 1$ times cheaper per step than full softmax and produces equivalent embeddings on intrinsic and extrinsic benchmarks.

The deeper lesson: *contrastive objectives* — distinguish positive pairs from negative pairs — are sufficient to recover useful geometry without a normalizing partition function. This insight underwrites CLIP (image-text), SimCLR (self-supervised vision), DPR (dense passage retrieval), and modern embedding models like BGE and E5. Negative sampling is the quiet ancestor of an enormous fraction of modern representation learning.

4.7 Subword tokenization: BPE, WordPiece, Unigram, SentencePiece

Static word embeddings are useless for out-of-vocabulary words. The fix that stuck — and that powers every modern LLM — is *subword* tokenization.

- **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016). Start with a character-level vocabulary; iteratively merge the most-frequent adjacent pair into a new token; repeat until the vocabulary size target is reached. Greedy, deterministic, simple. Used by GPT-2, GPT-3, LLaMA, Mistral. Tiktoken is a fast BPE implementation; cl100k_base is the OpenAI variant.
- **WordPiece** (Schuster & Nakajima, 2012; popularized by BERT). Similar to BPE but uses a likelihood-based merge criterion: at each step, merge the pair that most increases the language-model likelihood. Slightly more principled; WordPiece is the BERT/Roberta default.
- **Unigram language model** (Kudo, 2018). Treat tokenization as latent: start with a large vocabulary, iteratively prune subwords that least hurt the corpus likelihood. Allows multiple plausible tokenizations of the same string; SentencePiece's default. Used by T5 and mBART.
- **SentencePiece** is a *library*, not an algorithm. Implements BPE and Unigram with a unified interface; treats input as a raw byte stream (so it works on languages without explicit word boundaries like Chinese and Japanese). De facto standard for non-English-centric pretraining.

Tokenization choices have downstream consequences. A vocabulary that splits “tokenization” into [“token”, “ization”] makes the model effectively learn a compositional rule. A vocabulary that has “tokenization” as a single token forces it to memorize the word. The right tradeoff depends on data scale and language coverage. Frontier LLMs trend toward larger vocabularies (128k+ tokens) because token-savings on long inputs dominate the per-token embedding cost as V grows.

4.8 The linear hypothesis and analogies

The “king – man + woman $\approx$ queen” finding has been overclaimed and over-debunked. The current understanding:

- The linear analogy property is a *side effect* of the specific objective Word2Vec uses. Models trained with different objectives (LSA, GloVe, neural co-occurrence) display the property to varying degrees.
- The property is statistical, not deterministic. Many analogies fail; the famous successes are cherry-picked. Recovering analogies via $\arg \max_w \cos(\mathbf{w}, \mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}})$ requires excluding the source words from the candidate set; otherwise the math returns “king” itself with high probability.
- The linear structure exists *partly* because natural-language regularities have an additive flavor (gender, plurality, tense, royalty are roughly orthogonal axes), but mostly because the loss landscape is bilinear and the optimizer converges to a low-rank, smooth solution.

Static embeddings are now mainly of historical interest, but the “linear directions encode semantic features” idea has aged remarkably well. Probing modern LLM hidden states for *concept directions* (truthfulness, sentiment, entity types) frequently succeeds along approximately linear axes — which suggests that even contextualized representations preserve a residue of the geometric structure first observed in Word2Vec. The Linear Representation Hypothesis (Park et al., 2023) makes this precise for LLM internals.

Chapter 5. Recurrent Neural Networks

5.1 The recurrence equation

If we want unbounded history without exploding the parameter count, the obvious construction is a recurrence:

$$\mathbf{h}_t = \phi(W_h \mathbf{h}_{t-1} + W_x \mathbf{e}(x_t) + \mathbf{b}), \quad p(x_{t+1} \mid x_{1:t}) = \text{softmax}(W_o \mathbf{h}_t).$$

The hidden state $\mathbf{h}_t$ is a *summary* of the entire prefix. The same parameters are reused at every time step — perfect parameter sharing. This was the dominant LM architecture from roughly 2010 to 2017.

5.2 Backpropagation Through Time

Training requires unrolling the recurrence and applying backprop. Let $\mathcal{L}_t$ be the loss at step t . The gradient of $\mathcal{L}_T$ with respect to $\mathbf{h}_k$ for $k < T$ involves a product of Jacobians:

$$\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_k} = \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \prod_{t=k+1}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}.$$

Each factor is roughly $W_h^\top \cdot \text{diag}(\phi'(\cdot))$. The product of $T - k$ such matrices is the source of all RNN training pathology.

5.3 Vanishing and exploding gradients

Let $\sigma_{\max}(W_h)$ be the largest singular value. Then heuristically:

- If $\sigma_{\max} \cdot \|\phi'\|_\infty < 1$, the product shrinks geometrically. Long-range gradients **vanish**: distant losses cannot move early weights.
- If $\sigma_{\max} \cdot \|\phi'\|_\infty > 1$, the product grows geometrically. Gradients **explode**: training diverges.

This is the famous Pascanu/Mikolov/Bengio 2013 analysis. The set of W_h where gradients neither vanish nor explode is a measure-zero manifold, and SGD does not stay on it.

Practical patches helped a little:

- **Gradient clipping** (cap $\|\nabla\|$) tames explosion.
- **Orthogonal/identity initialization** of W_h delays vanishing.
- **Truncated BPTT** limits the chain to manageable length.

But the fundamental pathology remained.

5.4 Why recurrence was conceptually right but practically wrong

Recurrence is an *elegant* model of language: it commits to a single state vector that summarizes everything seen so far, like a Bayesian filtering posterior. But three properties of the recurrent computation are fatal at scale:

1. **Sequential compute.** $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$. Training cannot parallelize over time within a sequence. On modern accelerators, this leaves enormous FLOPs/sec on the floor.
2. **Information bottleneck.** All of history is squeezed through a fixed-dimensional vector $\mathbf{h}_t$. Long contexts overflow this bandwidth.
3. **Optimization fragility.** The vanishing/exploding behavior makes long-range credit assignment statistically unreliable.

The first is the one nobody could fix without leaving recurrence behind. Hardware was getting wider (more parallel) faster than it was getting deeper-in-time. Architectures that paralleled across the time axis would, eventually, win — even if their inductive bias was less elegant.

Chapter 6. LSTM and GRU

6.1 The Constant Error Carousel

If gradient products through nonlinear recurrences vanish or explode, the cleanest fix is to have a path along which the recurrence multiplier is *exactly* 1. That path is the **cell state** of the Long Short-Term Memory unit (Hochreiter & Schmidhuber 1997).

The trick: build a state $\mathbf{c}_t$ updated additively, not multiplicatively, gated by trainable functions. The “constant error carousel” (CEC) is the identity path along which information and gradient travel without decay.

6.2 LSTM equations

A standard LSTM cell, with σ for sigmoid and $\odot$ for elementwise multiply:

$$\begin{aligned}
 \mathbf{f}_t &= \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{forget gate} \\
 \mathbf{i}_t &= \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{input gate} \\
 \mathbf{o}_t &= \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) && \text{output gate} \\
 \tilde{\mathbf{c}}_t &= \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{candidate} \\
 \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{cell update} \\
 \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{hidden output}
 \end{aligned}$$

Read the cell update line carefully. When $\mathbf{f}_t \approx 1$ and $\mathbf{i}_t \approx 0$, $\mathbf{c}_t \approx \mathbf{c}_{t-1}$. The cell is a copy. Gradients flow backward through this copy without decay. That is the CEC.

6.3 GRU as a consolidation

Cho et al.'s Gated Recurrent Unit (2014) merges the forget and input gates into a single update gate $\mathbf{z}_t$ and exposes the full state directly:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$

Empirically, GRUs are competitive with LSTMs on most language tasks at lower parameter cost.

6.4 What gating fixed and what it didn't

Fixed:

- Vanishing gradients across hundreds of steps. LSTMs could carry information for thousands of tokens *if optimization cooperated*.
- Pathological short-term forgetting in vanilla RNNs.

Didn't fix:

- Sequential compute along the time axis — still inherent.
- Implicit information bottleneck — $\mathbf{c}_t$ is still a fixed-dim vector.
- Genuinely long-range, low-density signals (e.g., distant coreference). Gating biases the system toward keeping things, but offers no positional addressability.

LSTMs powered Google Translate (the first NMT production system), Siri's ASR backend, and countless ranking pipelines. They were good. They were not good enough at the scale the field was about to demand.

The lesson: **additive update paths beat multiplicative ones**. This single idea outlived RNNs entirely and lives on in residual connections — see Chapter 10.

Chapter 7. Sequence-to-Sequence Models

7.1 Encoder–decoder framing

Sutskever, Vinyals & Le (2014) framed translation as: read the source with one RNN, summarize it into a single vector, then generate the target with a second RNN conditioned on that vector.

$$\mathbf{c} = \mathbf{h}_{T_{\text{src}}}^{\text{enc}}, \quad p(y_t \mid y_{<t}, x) = \text{softmax}(\text{RNN}^{\text{dec}}(\mathbf{h}_{t-1}^{\text{dec}}, [\mathbf{e}(y_{t-1}); \mathbf{c}])).$$

This was a conceptual unification: any conditional sequence task (translation, summarization, dialogue, parsing) could be cast as seq2seq.

7.2 The bottleneck

The single context vector $\mathbf{c}$ is a hard ceiling on information. A sentence of 5 words and a paragraph of 500 words must pass through the same d -dimensional pipe. Even with $d = 1,024$, the channel capacity is fixed; the source information is not.

Empirically, BLEU scores on long sentences degraded sharply. A natural (and at the time radical) idea: **let the decoder reach back into the encoder's hidden states directly, instead of squeezing them through a single vector**.

That idea is attention.

7.3 Encoder-decoder architectures in detail

A canonical seq2seq model from the LSTM era looks like:

- **Encoder.** A multi-layer (often bidirectional) LSTM reads the input and produces a sequence of hidden states $\mathbf{h}_1^{\text{enc}}, \dots, \mathbf{h}_S^{\text{enc}}$. The final state, or a pooling thereof, becomes the *thought vector* $\mathbf{c}$.
- **Decoder.** A multi-layer LSTM initialized from $\mathbf{c}$, generating output tokens autoregressively conditioned on the encoder's summary.

The encoder and decoder are trained jointly by cross-entropy on output tokens; gradients flow from the output, through the decoder, through $\mathbf{c}$, into the encoder. The thought vector is whatever the model decides to put into a d -dimensional vector after seeing the source sentence — and there is no architectural pressure to make that vector well-structured.

Bidirectional encoders helped — the encoder sees the entire source, so $\mathbf{c}$ summarizes the full sentence rather than the last word — but did not eliminate the bottleneck. As source length grew, the gradient penalty for using $\mathbf{c}$ poorly stayed roughly constant while the cost of compressing more semantics into the same vector grew unboundedly.

7.4 Teacher forcing and exposure bias

A subtle issue in seq2seq training: at training time, the decoder receives the *true* previous token as input; at inference, it receives its *own* previous prediction. This mismatch is **teacher forcing's exposure bias**.

Symptoms: small errors compound at inference because the decoder has never been trained on its own off-distribution outputs. A single mis-generated token can derail the entire continuation, especially for long outputs.

Mitigations explored before the Transformer era:

- **Scheduled sampling** (Bengio et al. 2015). With probability ϵ during training, replace the true previous token with the model's own prediction. Gradually anneal ϵ . Improves robustness but introduces gradient bias because the sampling decision is non-differentiable.
- **Beam search at inference.** Keep multiple hypotheses; the best path may correct early mistakes by combining likelihoods of futures. Helps but does not fix the root cause.
- **Minimum-risk training and sequence-level losses.** Optimize a sequence-level metric (BLEU) directly via reinforcement learning. Hard to tune; high variance.

The exposure-bias problem persists in Transformers, partially masked by their scale and the strength of in-distribution generalization. Reasoning models (Chapter 19) revisit it: their reinforcement learning stages are essentially scheduled-sampling-on-steroids with a verifier replacing BLEU.

7.5 Beam search, length normalization, and coverage

For inference, beam search keeps the top- B hypotheses by accumulated log-probability and extends each by one token at a time. Three subtleties matter:

- **Length normalization.** Long sequences accumulate negative log-probability faster than short ones; without correction, beam search prefers terse outputs. The standard fix: divide log-probability by L^α for $\alpha \in [0.6, 0.9]$, a hyperparameter usually tuned on a held-out dev set.

- **Coverage penalty.** Without it, NMT models can drop content from the source. Wu et al. (Google’s NMT paper, 2016) added a penalty proportional to the maximum attention paid to each source position over all decoded steps, encouraging the decoder to “cover” every source word.
- **Beam-size diminishing returns.** $B = 5\text{--}10$ captures most of the quality gain; beyond $B \approx 20$ the model can produce *worse* outputs because beam search amplifies pathologies the model has at the head of its distribution. This counterintuitive result (Stahlberg & Byrne 2019) is one of the few clean empirical refutations of “more search is always better.”

Modern LLMs decode by sampling rather than beam search for most uses, because sampling produces more diverse outputs and avoids beam search’s degeneracy modes. Beam search persists in machine translation and structured generation tasks where the answer space is genuinely peaky.

7.6 BLEU, perplexity, and the metric problem

How do we know seq2seq is working? Several metrics, each with limitations:

- **BLEU** (Papineni et al. 2002). n -gram overlap between hypothesis and reference, with brevity penalty. Easy to compute, language-agnostic, weakly correlated with human judgment. The standard MT metric for two decades; criticized for rewarding surface overlap rather than semantic adequacy.
- **METEOR, chrF, TER.** Variations on BLEU with stem matching, character-level overlap, or edit distance.
- **BERTScore, COMET, BLEURT.** Learned metrics that compare hypothesis and reference in a contextualized embedding space. Better correlation with human judgment; more expensive to compute; carry their own model biases.
- **Perplexity on held-out target.** Measures how well the model assigns probability to the gold output, without sampling. Useful for training-curve diagnosis; not a generation-quality metric.

The honest summary: every automatic metric is an imperfect proxy for human judgment. BLEU was good enough to drive a decade of progress because it preserved *ordering* — a model with higher BLEU was usually better — even when its absolute numbers were not interpretable.

The architectural lesson: *measurement shapes the field*. Once attention raised BLEU on long sentences, the path was set. Had we been measuring something else (semantic faithfulness, factual accuracy), the architectural trajectory might have looked different. We are still paying for BLEU’s legacy in the sense that “next-token cross-entropy” remains our primary training objective even when the downstream tasks are reasoning or factuality, neither of which it directly measures.

Chapter 8. Attention Mechanisms

8.1 Bahdanau and Luong attention

Bahdanau, Cho & Bengio (2015) augmented the seq2seq decoder with an alignment mechanism. At each decoding step t , compute a score against every encoder state, normalize to weights, and produce a weighted sum:

$$e_{t,s} = a(\mathbf{h}_{t-1}^{\text{dec}}, \mathbf{h}_s^{\text{enc}}), \quad \alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}, \quad \mathbf{c}_t = \sum_s \alpha_{t,s} \mathbf{h}_s^{\text{enc}}.$$

The score a is a small MLP in Bahdanau's version, a dot product or bilinear form in Luong's. The decoder consumes $\mathbf{c}_t$ alongside its own state to produce the next token.

8.2 Path length and credit assignment

The deep reason attention worked is **path length**. Define the *effective path length* between an output at position t and an input at position s as the number of nonlinear hops the gradient must traverse. For an RNN, this is $|t - s|$. For attention, it is **constant** — one hop, regardless of $|t - s|$.

Short paths matter for two reasons:

1. **Gradient health.** Each hop is a chance for a Jacobian product to vanish, explode, or get stuck. Constant paths sidestep this.
2. **Credit assignment.** If the output at t depends on the input at s , attention can route gradient back to s in one step. The model can *learn* the dependency cleanly.

8.3 Attention as differentiable routing

Attention is, mathematically, a soft lookup. The query $\mathbf{q}$ asks a question; the keys $\mathbf{k}_s$ advertise what they answer; the softmax decides how strongly to listen to each value $\mathbf{v}_s$. The whole machine is differentiable, so the *routing* — the choice of who talks to whom — is itself learned.

This is conceptually different from convolution (fixed receptive field) and recurrence (sequential aggregation). Attention is a learned, content-dependent graph that changes per input. That property — content-based, dynamic connectivity — is the defining feature of the next chapter's architecture.

8.4 Why attention changed optimization dynamics

Once attention was added, three things happened:

- BLEU on long sentences stopped degrading.
- Training became more stable: gradients flowed through the attention shortcut.
- Models *learned alignment*, often mirroring linguistic alignment without supervision.

The next, obvious question: do we even need the recurrence anymore?

8.5 Additive vs multiplicative attention

Bahdanau attention uses an *additive* (a.k.a. concat) score:

$$e_{t,s} = \mathbf{v}_a^\top \tanh(W_a[\mathbf{h}_{t-1}^{\text{dec}}; \mathbf{h}_s^{\text{enc}}]).$$

Luong attention introduced two simpler alternatives:

$$e_{t,s}^{\text{dot}} = \mathbf{h}_t^{\text{dec}} \top \mathbf{h}_s^{\text{enc}}, \quad e_{t,s}^{\text{general}} = \mathbf{h}_t^{\text{dec}} \top W_a \mathbf{h}_s^{\text{enc}}.$$

Why does this distinction matter? Because of *implementation cost*. Additive attention requires a small MLP per (decoder, encoder) pair: $\mathcal{O}(T_{\text{dec}} \cdot T_{\text{enc}})$ MLP evaluations. Multiplicative attention reduces to a single matrix multiplication of decoder states with encoder states — one batched matmul, fully GPU-friendly.

Empirically, additive and multiplicative attention scored similarly when carefully tuned; multiplicative won decisively on speed at scale. The Transformer’s choice — scaled dot-product attention, which is multiplicative with a temperature normalization — is a direct descendant. The lesson: *fight for the fused matrix multiplication*. Architectures whose attention reduces to dense matmuls benefit from a decade of accelerator co-design that custom layers do not.

8.6 Query-key-value as a generalization

Bahdanau’s formulation conflated keys and values: the same encoder hidden state was used both for scoring and for being averaged. The Transformer separates them:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Now the model can learn to *advertise* one feature (in K) for matching while *delivering* a different feature (in V) when matched. A token that is “the subject of the sentence” might match queries asking “who acts?” via its key, but provide its name and grammatical role via its value.

The key/value separation is a small change with outsized effect. It is the difference between asking “look at this feature” and “look at this feature, but if you do, here is what you’ll get.” Most architectural elaborations of attention since 2017 — encoder-decoder cross-attention, retrieval attention, Perceiver IO — reuse this three-way decomposition because it exposes orthogonal degrees of freedom that the model can use independently.

8.7 Attention as a graph

Attention can be visualized as a bipartite graph: queries on one side, keys/values on the other, weighted edges given by softmax. In self-attention, the same set of tokens plays both roles, giving a complete weighted directed graph at each layer.

Three useful graph-theoretic readings:

- **Each layer redraws the graph.** The graph at layer $\ell + 1$ is conditioned on the representations produced at layer ℓ . Information can travel through any sequence of edges; if a graph has a path from token i to token j across L layers, j can learn about i .
- **Heads are parallel graph copies.** Multi-head attention runs several independent graphs simultaneously, each computing a different view of the relationships between tokens.
- **Sparsity is graph pruning.** Local attention, sparse attention, sliding-window attention all amount to fixing the graph topology in advance and computing only over its edges. The trade-off is precisely the trade-off between dense and sparse graphs in classical graph algorithms.

This lens is helpful when comparing architectures: a Transformer is a dynamic, content-dependent fully-connected weighted directed graph; an SSM is a constrained DAG with weight-tied edges; a CNN is a fixed translation-invariant sparse graph. Each architecture is a different *prior over what information flows are easy* — and that prior is what the bias-variance tradeoff of training plays against.

Chapter 9. The Transformer

9.1 The provocation

“Attention Is All You Need” (Vaswani et al., 2017) answered: no, we do not need recurrence. Replace it entirely with attention, stack residual blocks, and let the GPUs sing.

The resulting architecture became the default for nearly every sequence task within three years. The reasons are partly mathematical, mostly systems-level, and slightly historical. We unpack each.

9.2 Self-attention math

Given a sequence of token representations $X \in \mathbb{R}^{T \times d}$, project to queries, keys, and values:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

then compute

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

The $\sqrt{d_k}$ scaling is essential: dot products in d_k -dimensional space have variance $\propto d_k$, which would push the softmax into one-hot territory. Dividing keeps gradients alive.

A *causal* mask prevents attending to future positions:

$$M_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases}, \quad \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

9.3 Multi-head attention

A single softmax is one viewpoint. Multi-head attention runs h smaller attention operations in parallel and concatenates:

$$\text{head}_i = \text{Attn}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)}),$$

$$\text{MHA}(X) = [\text{head}_1; \dots; \text{head}_h]W_O.$$

Heads tend to specialize — syntactic heads, positional heads, copy heads, induction heads — though specialization is emergent and noisy. The point is that h heads of dimension d/h have the same FLOPs as one head of dimension d but h different attention patterns simultaneously.

9.4 The residual stream

Each Transformer block has the structure

$$X \leftarrow X + \text{MHA}(\text{Norm}(X)),$$

$$X \leftarrow X + \text{MLP}(\text{Norm}(X)).$$

The right way to read this is: the activations are a **residual stream**, a per-position bus on which every layer can read and write. MHA reads from the stream, computes a contribution, and writes it back. MLP does the same. Layers do not transform; they *add*.

This shifts how we think about the network. There is no “first hidden representation, second hidden representation, ...”. There is one stream, with L layers’ worth of contributions accreted onto it. Mechanistic interpretability (induction heads, attention heads as circuits) builds on exactly this model.

9.5 Pre-Norm vs Post-Norm

The original paper used **post-norm**:

$$X \leftarrow \text{LN}(X + \text{MHA}(X)).$$

This is unstable at depth: the residual path is interrupted by a normalization, so gradients are not clean. Almost every modern LLM uses **pre-norm**:

$$X \leftarrow X + \text{MHA}(\text{LN}(X)).$$

The residual path is now an unobstructed identity. Training scales to hundreds of layers without exotic warmup tricks. Pre-norm is a numerical, not theoretical, change — and yet it unlocked depth.

9.6 Positional encodings

Self-attention is permutation-equivariant: swap two tokens, and the outputs swap correspondingly. To inject order, add positional information.

- **Sinusoidal** (original Transformer): $\text{PE}(t, 2i) = \sin(t/10000^{2i/d})$, $\text{PE}(t, 2i + 1) = \cos(\cdot)$. Allows extrapolation in principle, in practice mediocre.
- **Learned absolute**: a free embedding per position. Simple, doesn’t extrapolate.
- **Relative positional bias** (T5, Shaw et al.): bias added to attention logits as a function of $|i - j|$.
- **RoPE** (Su et al., 2021; covered in Chapter 14): rotate query/key vectors by an angle proportional to position. Encodes *relative* position via the inner product structure. Now standard.
- **ALiBi** (Press et al., 2021): a linear bias on attention logits proportional to distance. Cheap, extrapolates surprisingly well.

The choice matters more than people initially thought, especially for long-context generalization.

9.7 Why Transformers scale: complexity analysis

Consider a Transformer with L layers, model dimension d , sequence length T , vocabulary V .

Resource	Cost
Self-attention FLOPs / layer	$\mathcal{O}(T^2 d)$
MLP FLOPs / layer	$\mathcal{O}(T d^2)$

Resource	Cost
Parameters (excl. embeddings)	$\mathcal{O}(Ld^2)$
Activations memory	$\mathcal{O}(LTd)$
KV cache (inference)	$\mathcal{O}(LTd)$

Two regimes follow:

- $T \ll d$ (short sequences, big models): MLP dominates; compute is matmul-heavy; perfect for accelerators.
- $T \gg d$ (long sequences): attention's T^2 dominates; this is the long-context bottleneck of Chapter 16.

Crucially, the time dimension is **fully parallel**. Within a sequence, every position's representation at layer ℓ can be computed in parallel given layer $\ell - 1$. Compare with RNNs: zero parallelism along T . On a GPU, this is a 100× difference, not a 2× difference.

9.8 Why Transformers won

- **Hardware fit.** Dense matmuls are what GPUs/TPUs are built to do.
- **Gradient health.** Residual streams and short attention paths keep gradients well-behaved at depth.
- **Inductive bias appropriate to language.** Long-range, content-addressed dependencies are exactly what attention models well.
- **Modular composition.** Stacking blocks Just Works.

What it traded: dense T^2 attention, dense parameters per layer, and an unwieldy KV cache at inference. We are still paying for these, in different ways, in every chapter that follows.

9.9 The softmax: why this nonlinearity?

Self-attention's softmax does three things at once. First, it normalizes — outputs sum to 1, so attention is a probability distribution over keys. Second, it sharpens — the exponential pulls mass toward the largest logit. Third, it is differentiable everywhere, with non-zero gradient on every input.

Each of these has been challenged. Linear attention (Chapter 15) drops the softmax; sparsemax replaces it with a piecewise-linear projection onto the simplex; sigmoid attention (one logit at a time, no normalization) has been tried; multiple-softmax decompositions exist. None has displaced standard softmax in production-scale LLMs. The reasons are subtle but consequential:

- **Implicit competition.** Softmax's Jacobian is $\partial\pi_i/\partial z_j = \pi_i(\delta_{ij} - \pi_j)$, which couples every output to every input. Increasing one logit *necessarily* decreases the relative weight of the others. This produces a routing dynamic where tokens compete for attention; linear attention has no such coupling and consequently learns less crisp routings.
- **Saturating sharpness.** When the model is confident — one logit is much larger than the rest — softmax saturates and gradients on the unselected keys vanish. This is *desirable*: the optimizer stops pushing on already-decided routing, freeing capacity for ambiguous queries.
- **Numerical stability.** Subtracting the max ($z_i \rightarrow z_i - \max_j z_j$) makes softmax stable in fp16/bf16 across the wide range of logit magnitudes that arise during training. The same identity is the kernel of the FlashAttention online softmax (§15.6).

Temperature is the same idea read differently: $\text{softmax}(z/\tau)$. As $\tau \rightarrow 0$, attention becomes hard argmax (no gradient, no learning). As $\tau \rightarrow \infty$, attention is uniform (no information). The $1/\sqrt{d_k}$ scaling in standard attention is an implicit temperature: without it, dot products in d_k -dimensional space would have variance proportional to d_k , pushing the softmax into the saturated regime at random initialization and stalling learning.

9.10 Gradient flow through self-attention

Why do Transformers train at depth in a way prior architectures could not? The answer is in the Jacobians.

For a residual block $X \leftarrow X + f(\text{LN}(X))$ — with f either MHA or MLP — the Jacobian with respect to the input is

$$\frac{\partial X_{\text{out}}}{\partial X} = I + \frac{\partial f}{\partial \text{LN}(X)} \cdot \frac{\partial \text{LN}(X)}{\partial X}.$$

The identity matrix on the left is the residual highway. Even if the second term has spectral norm $\ll 1$ — as it typically does in well-initialized networks — the identity ensures gradients propagate across L layers without vanishing. Compare with a vanilla MLP, whose Jacobian is *just* the second term: across $L = 96$ layers, gradients shrink by a product that almost certainly underflows fp16.

The MHA Jacobian itself decomposes through the softmax. For attention output $Y = \text{softmax}(QK^\top / \sqrt{d_k})V$, the gradient with respect to V is the attention matrix transposed; the gradients with respect to Q and K involve the attention matrix and its softmax derivative. The crucial property: every position is in every other position's gradient path. Credit assignment is genuinely $\mathcal{O}(1)$ in path length, which is why long-range dependencies can be learned at all.

What can still go wrong even with this favorable structure:

- **Logit explosion.** If $QK^\top$ grows during training, softmax saturates and gradients on most keys vanish — locally, optimization stalls. Modern stacks use QK normalization (e.g., dividing Q and K by their norms before the dot product) to bound logits.
- **Activation drift.** Without normalization, the residual stream's magnitude grows monotonically with depth as more block contributions are added. Pre-norm placed inside each block keeps the stream bounded relative to itself.
- **Attention entropy collapse.** Heads can become near-deterministic early in training, learning slowly thereafter. Empirically rare in well-tuned stacks, but a documented failure mode in certain initialization regimes.

9.11 Attention head specialization: a brief catalog

Mechanistic interpretability has uncovered that, despite training with no inductive bias toward division of labor, attention heads in trained Transformers fall into a small number of recognizable types. A non-exhaustive catalog:

- **Previous-token heads.** Attend almost exclusively to position $t - 1$. These appear in early layers and act as a one-token shift register, useful for local syntax.

- **Induction heads** (Olsson et al. 2022). A two-head circuit: one head copies tokens from prior occurrences of a matching prefix; another reads from there. This is the mechanism behind few-shot in-context learning’s basic copy/repeat behavior.
- **Punctuation / boundary heads**. Attend to the previous period, comma, or newline. Useful for sentence-level segmentation and discourse structure.
- **Name-mover heads** (Wang et al. 2023, IOI circuit). Move named entities from earlier mentions to later positions in a structured way; the basis of indirect-object identification circuits.
- **Suppression heads**. Push down the logits of recently-emitted tokens, implementing an anti-repetition prior.
- **Successor heads**. Increment ordinal patterns (“Monday” → “Tuesday”, “2” → “3”).

The point is not the catalog. It is that *the residual-stream view of Transformers is operationally useful*. Layers do not transform monolithically; they read, compute, and write back. The result is a trained network with locally interpretable circuits — a property that is rare among neural architectures and that has consequences for safety, debugging, and even architecture design (future work can deliberately reserve channels in the residual stream for known circuit families).

9.12 Initialization, width, depth, and μP

A Transformer with L layers, width d , and h heads has many ways to fail to train. The most subtle failure is *hyperparameter drift with scale*: the learning rate that works at $d = 256$ may catastrophically diverge at $d = 8192$.

The classical solutions are scaling rules: scale the residual branches by $1/\sqrt{L}$ (or sometimes $1/L$) at initialization so that the residual stream’s variance does not blow up at depth; initialize attention/MLP weights with variance $\propto 1/\text{fan_in}$, possibly with a depth-dependent factor.

A more principled answer is **Maximal Update Parameterization** (μP , Yang & Hu 2021). The idea: choose initialization scales, learning rates, and output multipliers so that, in the infinite-width limit, every layer’s features change by an order-1 amount per step and the network’s output changes by an order-1 amount per step. The practical consequence is striking: hyperparameters tuned at small width transfer to large width with minimal adjustment. Train at $d = 256$ to find the right learning rate; reuse it at $d = 8192$ at hundreds-of-millions-of-dollars scale.

μP is not yet universally adopted (some training stacks still use ad hoc scaling that empirically works), but it is the right mental model. Architectures that scale gracefully are not an accident; they are *designed* so that the operator norms of per-layer transformations remain stable across width and depth, and so that the optimizer’s effective step size in feature space is invariant to the network’s size.

Chapter 10. Decoder-Only LLMs

10.1 The GPT lineage

OpenAI’s GPT series stripped seq2seq down to a decoder-only Transformer with causal masking. There is no encoder; there is no separate “context” and “target”; there is only one sequence and the next token to predict.

- **GPT (2018):** 117M parameters, 12 layers, $d = 768$. Demonstrated transfer from unsupervised LM pretraining to supervised fine-tuning.
- **GPT-2 (2019):** up to 1.5B parameters. Showed that a sufficiently large LM does many tasks zero-shot.
- **GPT-3 (2020):** 175B parameters. Demonstrated **in-context learning** at scale.
- **GPT-4 (2023):** scale undisclosed; multimodal, instruction-tuned, RLHF-aligned.

Subsequent open-weight LLMs (LLaMA, Mistral, Qwen, DeepSeek, Gemma, Falcon, Phi, MPT, Pythia, OLMo) all follow roughly the same template: decoder-only Transformer, causal LM, RoPE, RMSNorm, SwiGLU, GQA.

10.2 Why causal language modeling dominates

The core argument:

1. **Universality.** The chain rule is exact; there is no information-theoretic loss in factorizing.
2. **Generative.** Sampling is straightforward; no need for a separate decoder.
3. **Self-supervised.** Every token in a corpus is a label.
4. **Inference-friendly.** Causal attention permits a *KV cache*: to generate token $T + 1$, you do not recompute over $1 \dots T$; you append.
5. **Scaling-friendly.** Encoder-decoder splits compute artificially; decoder-only spends all FLOPs on one homogeneous stack, simplifying parallel training.

There are languages and tasks where bidirectional encoders (BERT, T5) dominate small-scale supervised benchmarks. But once scale is sufficient, decoder-only catches up, and its inference and generation properties dominate the practical cost-benefit calculus.

10.3 In-context learning as emergent

GPT-3 showed something genuinely surprising: with no parameter updates, just prompts containing input/output exemplars, the model performs new tasks. Few-shot performance scales with model size in a way that few-shot performance of fine-tuned models does not.

The interpretation: large enough autoregressive models internalize a **meta-learning algorithm**. Olsson et al. (2022) localized one mechanism — *induction heads* — pairs of attention heads that copy patterns. There are surely more. The mechanistic story is incomplete; the phenomenon is robust.

The practical consequence: a sufficiently large LM is a general-purpose function approximator over text. Tasks become prompts. Fine-tuning becomes optional. The locus of customization shifted from “train a new model” to “write a better prompt.”

10.4 What decoder-only didn’t fix

- $\mathcal{O}(T^2)$ attention.
 - KV cache memory growing with context length.
 - No principled separation of “thinking” from “outputting” — every reasoning step must fit in the forward pass or in the generated tokens. Chapter 20 returns to this.
-

Chapter 11. Pretraining Paradigms

11.1 The three families

Family	Objective	Examples
CLM	Predict next token left-to-right	GPT, LLaMA, Mistral
MLM	Predict masked tokens given full context	BERT, RoBERTa
Encoder-decoder denoising	Reconstruct corrupted span	T5, BART

Each has a different training/inference profile.

11.2 MLM vs CLM

MLM (masked language modeling). Mask 15% of tokens; predict them from bidirectional context. Strong representations for *understanding* tasks (classification, NER, retrieval). But: the model never learns to *generate* sequences; the train/test distribution mismatch (mask token absent at inference) requires careful handling.

CLM (causal language modeling). No masking; predict every token from preceding tokens. Strong for *generation*. Naturally produces a valid sampler.

The empirical lesson, post-2020: **at scale, CLM gives you both**. Large CLMs do classification well via prompting; they do generation natively; they do retrieval (via embedding heads or last-layer pooling) competently. MLM models do generation poorly and require auxiliary heads.

11.3 Denoising and span corruption

T5's "everything is text-to-text" approach trains on span-corruption: replace spans with sentinel tokens, learn to fill in. Useful inductive bias: forces the model to learn long-range structure. But the encoder-decoder split means roughly half the parameters are used for either reading or writing at any given step — inefficient at high scale.

11.4 Why decoder-only won at scale

Three reasons, in roughly increasing importance:

1. **Inference economics.** KV-cached autoregression amortizes attention over the prefix. Encoder-decoder pays full attention cost per decode step.
2. **Compute homogeneity.** A single stack, single objective. Training and serving are simpler; compiler optimization is cleaner.
3. **Empirical generality.** Once at scale, prompting gives MLM-style understanding *for free*; the converse is false.

This is constraint reallocation: we pay a small price in understanding-task efficiency at small scale, in exchange for inference simplicity, generative capability, and clean scaling curves.

11.5 Data mixtures and curricula

Pretraining data is not a homogeneous corpus. It is a deliberately *mixed* and often *staged* assembly of sources: web crawl, books, code, scientific text, math, reference works, multilingual content. The mixture is itself an architectural decision — possibly the most underrated one.

A well-known finding (Du et al. 2022; Xie et al. 2023, *DoReMi*): the mixture that minimizes training loss on the mixture is *not* the mixture that maximizes downstream capability. Pretraining on too much code helps code benchmarks at the cost of general reasoning; too much general web data helps perplexity but hurts code, math, and instruction-following.

The dominant pattern in 2023–2025 open models:

- **Heavy web crawl** (50–70%), filtered aggressively by quality classifiers (FineWeb-Edu, DCLM-Pool, RedPajama-V2). Filtering matters more than raw quantity.
- **Code** (10–25%), often weighted higher than its proportion in raw text would suggest. Code teaches structured reasoning and seems to transfer to math and general logic.
- **Math** (5–10%), including ArXiv, math-forum data, OpenWebMath, and synthetic chain-of-thought.
- **Books** (5–10%), for long-form coherence and rare vocabulary.
- **Multilingual** (10–30%, depending on target), for cross-lingual generalization.
- **Curated high-quality** (1–5%), often used in a final-phase upweighting.

Curriculum. Many recent training runs use a multi-phase schedule: an early phase on broad noisy data, a mid phase with balanced mixtures, and a late phase that upweights high-quality, instruction-style, or domain-specific data. Phi-3, MiniCPM, and DeepSeek-V3 all describe variants. The hypothesis: capacity allocated late in training is allocated to whatever is in the late-stage mixture, so curate that stage carefully. A specific recipe gaining traction is “Warmup-Stable-Decay” (WSD) where the learning-rate decay phase coincides with a high-quality data phase, fusing optimization curriculum with data curriculum.

The architectural implication: *the data pipeline is part of the model*. Two LLaMA-3-70B-architecture models trained on different data mixtures are different models. Reproducibility, ablation, and cross-paper comparison are correspondingly hard — and the field’s tendency to credit architecture for what is actually data engineering is a persistent source of confusion.

11.6 Fill-in-the-middle for code

Causal language modeling sees only the past. For code completion in IDEs, the user wants completions that fit *between* an existing prefix and suffix. Naïve CLM cannot do this; it only knows the prefix.

Fill-in-the-middle (FIM) training (Bavarian et al. 2022) reformats some training examples as

[PRE] prefix [SUF] suffix [MID] middle

so that the model still learns left-to-right but on permuted documents. At inference, the user assembles a prompt of the same form and the model generates the missing middle. This is a tiny modification to the data pipeline that unlocks an entire UX class.

The architectural lesson: *the pretraining objective is a contract about what the model can do at inference*. Causal LM trains a left-to-right text generator; FIM extends the contract to bidirectional infilling without changing a single weight in the model. Almost every modern code model (StarCoder, DeepSeek-Coder,

CodeLlama, Qwen-Coder) trains with FIM.

A subtler version: span corruption with multiple infill spans. T5's pretraining objective prefigured this; FIM brings it back inside a decoder-only stack with minimal changes to the data format.

11.7 Instruction tuning and RLHF: where do they fit?

The classical view: pretraining is unsupervised, instruction tuning is supervised fine-tuning (SFT), and RLHF is the cherry on top. Architecturally, all three operate on the same Transformer with the same parameters; they differ only in objective and data.

A more useful framing: instruction tuning and RLHF are *continuations* of pretraining. They are different objectives but on the same parameter manifold; they steer the model's distribution toward useful behaviors without changing what the model fundamentally is. The only architectural feature they require is one that pretraining must already provide: a capable base model with sufficient generalization headroom.

This explains a curious empirical fact: instruction-tuning a small model often produces something less capable than instruction-tuning a large one *with the same instruction data*. Instruction tuning is not "teaching" the model the tasks; it is *unlocking* representations the pretrained model already had. Pretraining is where capability lives; post-training is where it is shaped.

The constraint reallocation: post-training shifts compute from broad-distribution pretraining to narrow-distribution fine-tuning. The cost is potential overfitting to the instruction distribution; the benefit is alignment to user-facing behaviors. RLHF/DPO/GRPO further shift compute from fitting demonstrations to optimizing against preferences, which can produce gains the SFT data alone cannot reach — at the price of reward hacking and capability narrowing if the preference model is mis-specified.

11.8 Repetitions, deduplication, and the data wall

A subtle but important variable in pretraining is *how many times the model sees each token*. With Chinchilla scaling, a 70B model sees roughly 1.4T tokens once each. Beyond that:

- **More tokens, fewer repetitions.** Requires more unique high-quality data, which is now scarce — Hoffmann et al. and later analyses estimate the pool of high-quality web text at roughly 10^{13} tokens, with the genuinely-good fraction much smaller.
- **Same tokens, more repetitions.** Repetition factor up to $\sim 4\times$ is roughly equivalent to training on that many fresh tokens (Muennighoff et al. 2023); beyond $\sim 4\times$, returns diminish steeply, with diminishing-then-negative returns past $\sim 16\times$.
- **Synthetic data.** Generate tokens with a strong model, filter with a verifier, train on the result. Phi, MiniCPM, and others have demonstrated impressive small-model results this way; the scalability to frontier scale is an open empirical question.
- **Deduplication.** Removing exact and near-duplicates from the corpus is one of the most reliable quality wins. A poorly deduplicated corpus *forces* the model to memorize boilerplate; a clean corpus directs capacity toward genuine signal.

For models trained well past Chinchilla (LLaMA-3 at ~ 100 tokens/parameter, MiniCPM at much higher), the question is how much of the gain is real generalization and how much is memorization of the few-times-seen tail. The answer, from membership-inference and influence-function studies, is a mix: real gains exist, but memorization is non-trivial and copyright/privacy concerns become pressing.

The architectural implication: as we approach the data wall, *what we train on* becomes the binding constraint, not how. New architectures matter less than new data sources, synthetic-data quality, or self-improvement loops that generate their own data. Several research programs (process supervision, expert iteration, RL with verifiable rewards) are best understood as data-acquisition strategies dressed up as training objectives.

Chapter 12. Scaling Laws

12.1 The Kaplan scaling laws

Kaplan et al. (2020) measured loss as a function of compute C , parameters N , and data D , finding power-law relationships:

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}.$$

The exponents α were small (around 0.05–0.1), implying: doubling everything halves loss only modestly, but progress is sustained over many orders of magnitude. Crucially, the laws were **smooth** — no thresholds, no plateaus over the range studied.

12.2 The Chinchilla correction

Hoffmann et al. (2022) revisited the laws and found that, for a fixed compute budget C , the **compute-optimal** allocation between N and D is not what Kaplan recommended. Their law, in the simplest form:

$$L(N, D) \approx E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

with $\alpha \approx \beta \approx 0.5$. Setting $\partial L / \partial N = \lambda \partial L / \partial D$ at fixed $C \approx 6ND$, the optimum has $D \propto N$. In practice: about **20 tokens per parameter**. Kaplan’s models were severely under-trained. Chinchilla (70B params, 1.4T tokens) outperformed Gopher (280B, 300B tokens) at one quarter the parameters.

12.3 Compute-optimal vs inference-optimal

Chinchilla scaling minimizes *training loss per training FLOP*. But most LLMs serve far more inference than training tokens. From an *inference* perspective, smaller-but-overtrained models are better: they cost less per query forever. LLaMA, Mistral, and similar open models train past Chinchilla-optimal — sometimes 100+ tokens per parameter — because they will be deployed billions of times.

This is the cleanest example of constraint reallocation in the modern era: the optimal point depends on which axis (training cost, inference cost, latency) is the binding constraint.

12.4 What scaling laws imply for architecture design

Scaling laws are *humbling*. Many architectural innovations that look like clear wins at small scale evaporate at 10^{23} FLOPs. The community’s lesson:

- **Run the scan.** If your innovation does not improve the scaling exponent or the prefactor, it is not improving anything that will matter.
- **Ablate at multiple scales.** A change that helps at 100M may hurt at 70B.
- **Reason about residual error.** Most improvements reduce A or B (prefactors) not α or β (exponents). Real exponent changes are rare and revolutionary.
- **Watch for the floor E .** Once you are within ϵ of the irreducible loss, you cannot improve further by scaling alone. New objectives or new data are required.

12.5 Deriving the Chinchilla optimum

A clean derivation makes the Chinchilla finding more memorable than the slogan “20 tokens per parameter.”

Start from the loss decomposition

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

with E the irreducible (data-entropy) floor, N parameters, D training tokens, and $\alpha, \beta > 0$ empirically estimated. Compute is $C \approx 6ND$ for dense Transformer training (the constant 6 captures forward, backward, and weight-update FLOPs per token per parameter under standard accounting).

Minimize L subject to fixed C . Lagrangian:

$$\mathcal{L}(N, D, \lambda) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} - \lambda(6ND - C).$$

First-order conditions:

$$\frac{\partial \mathcal{L}}{\partial N} = -\alpha A N^{-\alpha-1} - 6\lambda D = 0, \quad \frac{\partial \mathcal{L}}{\partial D} = -\beta B D^{-\beta-1} - 6\lambda N = 0.$$

Eliminate λ :

$$\frac{\alpha A}{N^{\alpha+1}} \cdot \frac{1}{D} = \frac{\beta B}{D^{\beta+1}} \cdot \frac{1}{N} \implies \frac{\alpha A}{\beta B} = \frac{N^\alpha}{D^\beta} \cdot \frac{N^0}{D^0} \dots \implies \frac{D^\beta}{N^\alpha} = \frac{\beta B}{\alpha A}.$$

For Chinchilla’s measured $\alpha \approx \beta \approx 0.34$ and A, B of comparable magnitude, this implies the optimum lies on the line $D \propto N$, with the proportionality constant landing in the neighborhood of 20 in their fit.

Two non-obvious morals:

- The optimum is a *line* in (N, D) space, not a point. As compute C grows, you slide along that line; the right N and the right D both grow as roughly $\sqrt{C}$.
- The exponents α, β are what determine the slope of the line. Architectures or data regimes with materially different exponents would imply *very different* optimal allocations. “20 tokens per parameter” is a property of contemporary Transformer training on web-scale text, not a universal constant.

12.6 What scaling laws do not predict

The clean scaling laws hide several inconvenient truths that any practitioner needs to internalize.

- **Data quality.** D is *training tokens*, not *information content*. A trillion tokens of low-quality web crawl is not equivalent to a trillion curated tokens. Recent work (FineWeb, DCLM, RefinedWeb, Dolma) shows that data filtering changes the prefactor B by 20–40% — an effect comparable to a 4× compute increase, achieved without any architectural change.
- **Distributional shift.** Scaling laws are measured on training-distribution loss. Out-of-distribution loss can scale very differently and sometimes barely at all.
- **Downstream task accuracy.** Loss scales smoothly; accuracy on a particular task often does not. Specific capabilities can appear suddenly, plateau, or even decline as scaling continues.
- **Capability emergence.** Tasks that produce zero score below a compute threshold and non-zero score above it look like phase transitions. Whether they reflect real qualitative shifts or artefacts of binary metrics is the subject of §12.7.
- **Architectural changes.** Switching from dense to MoE, or from attention to SSMS, changes the constants and possibly the exponents. Scaling laws are *architecture-conditional*; comparing across architectures requires fitting separate laws per architecture.
- **Inference-time tricks.** Chain-of-thought, best-of-N, tool use, and verifier-driven search can multiplicatively change a model’s effective task capability without changing N , D , C at all. A scaling law over forward-pass loss is silent on this.

The honest framing: scaling laws are excellent predictors of training loss, mediocre predictors of task accuracy, and silent on safety, factuality, and reasoning depth. A research program that treats scaling laws as the only metric will optimize for the wrong target.

12.7 Emergence: phase transition or measurement artefact?

Wei et al. (2022) and others observed that many tasks exhibit *emergent* behavior: zero or near-zero performance below a compute threshold, then a rapid rise. This drove a wave of speculation about phase transitions in capability and discontinuous generalization.

Schaeffer, Miranda, and Koyejo (2023) pushed back hard. Many emergence curves, they argued, are artefacts of the metric. Exact-match accuracy is binary (right/wrong); replace it with a continuous metric (token probability of the answer, edit distance to the answer, BLEU), and the same data shows a smooth power-law improvement. Under that view, the underlying loss decreases smoothly; the *thresholding* of the evaluation creates the appearance of emergence.

Both can be true simultaneously, and the picture is now more nuanced:

- Some apparent emergence is metric-induced: thresholding a smooth curve at a stringent cutoff makes a kink look like a wall.
- Some emergence looks like a real qualitative shift, particularly for tasks that require chaining of subroutines (long multiplication, multi-hop reasoning). The probability of getting *all* substeps right at once does scale super-linearly with the probability of getting one substep right.
- Compositional capability — solving a task by combining several already-mastered subtasks — shows the most genuine sharp transitions, and these are not just artefacts.

The architectural lesson: *do not rely on smooth loss curves to predict task behavior, and do not rely on binary task accuracy to predict the underlying capability gradient*. Build evaluations that are continuous and granular; build a stack of evaluations that probe substeps as well as full tasks. The model's loss curve is an opinion poll on the average sample; the user cares about the worst-case sample.

Chapter 13. Optimization & Training Stability

13.1 The residual stream as a gradient highway

Every Transformer layer is $X \leftarrow X + f(X)$. The Jacobian of the layer with respect to its input is $I + f'(X)$. The gradient through L stacked layers is roughly $\prod_{\ell=1}^L (I + f'_\ell)$. As long as $\|f'_\ell\|$ is small, this product behaves like I plus a sum, not a runaway product. **Additive updates do not vanish or explode.**

This is why depth scales smoothly in Transformers but brutally in old plain MLPs. Residual streams are the structural reason GPT-3's 96 layers train at all.

13.2 LayerNorm and RMSNorm

LayerNorm normalizes per token across the feature dimension:

$$\text{LN}(x) = \frac{x - \mu}{\sigma} \odot \gamma + \beta, \quad \mu = \frac{1}{d} \sum_i x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_i (x_i - \mu)^2}.$$

RMSNorm (Zhang & Sennrich, 2019) drops the centering:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2}} \odot \gamma.$$

Two reasons RMSNorm is now standard:

1. **Cheaper.** No mean computation, no bias term. Saves a small but non-trivial fraction of activation FLOPs and parameters.
2. **No worse, often better.** Empirically, removing the mean does not harm and sometimes helps. The signal in the residual stream is roughly zero-mean already.

13.3 SwiGLU and MLP geometry

The original Transformer MLP was $\text{ReLU}(xW_1)W_2$. Modern stacks use a **gated linear unit** with Swish activation:

$$\text{SwiGLU}(x) = (\text{Swish}(xW_1) \odot xW_2)W_3, \quad \text{Swish}(z) = z \cdot \sigma(z).$$

The gate $\sigma(z)$ multiplies a candidate by a per-feature, content-dependent value in $(0, 1)$. Conceptually: every MLP unit has a tiny built-in attention-over-itself. SwiGLU consistently outperforms plain ReLU/Gelu at matched parameter count.

13.4 Rotary Position Embeddings

RoPE encodes position by **rotating** query and key vectors in 2D subspaces. For a head dimension split into pairs, the rotation angle at position t is $t \cdot \theta_i$ for the i th pair, with $\theta_i = 10000^{-2i/d_k}$. The inner product $\langle R_t \mathbf{q}, R_s \mathbf{k} \rangle$ depends only on $t - s$:

$$\langle R_t \mathbf{q}, R_s \mathbf{k} \rangle = \langle \mathbf{q}, R_{s-t} \mathbf{k} \rangle.$$

Properties RoPE delivers cleanly:

- Pure relative position dependence inside attention.
- No extra parameters.
- Smooth extrapolation properties (with adjustments — NTK-aware, YaRN).
- Easy to swap into existing Transformer code.

RoPE is now the default position encoding in most modern open LLMs.

13.5 GQA: grouped-query attention

Multi-head attention has h key/value pairs per layer. At inference, the KV cache stores $\mathcal{O}(L \cdot T \cdot h \cdot d_k)$ floats. For long contexts and many heads, this is most of the GPU memory.

Multi-query attention (Shazeer 2019) collapses to one shared K and V across all heads. Memory savings are $h \times$. Quality drops slightly.

Grouped-query attention (GQA, Ainslie et al. 2023) is the compromise: g groups of heads share K and V within each group. With $g = 8$ for $h = 64$, memory drops $8 \times$ and quality is essentially unchanged. GQA is the default in LLaMA-2-70B, LLaMA-3, Mistral, and most modern serving stacks.

The pattern across this chapter: **modern LLM “tricks” are not heuristics; they are surgical reductions of memory and bandwidth bottlenecks identified by careful profiling.**

13.6 Optimizers: AdamW and decoupled weight decay

LLMs train with AdamW almost universally. The Adam update is

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$
$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

with $\hat{m}_t, \hat{v}_t$ bias-corrected estimates of the first and second gradient moments and $\beta_1 \approx 0.9, \beta_2 \in [0.95, 0.999]$ standard. The per-parameter rate $\eta / \sqrt{\hat{v}_t}$ is what handles the wildly different scales between embedding parameters, attention weights, and MLP weights.

Why v_t ? Empirically, the gradient signal in Transformers has heavy-tailed second moments — a few coordinates have huge gradients, most have tiny ones, and the disparity changes across training. SGD with one global learning rate cannot handle this dynamic range. Adam’s normalization equalizes effective steps across coordinates and absorbs the coordinate-wise curvature without computing a Hessian.

AdamW (Loshchilov & Hutter 2019) decouples weight decay from the gradient. Standard L2 regularization adds $\lambda\theta$ to the gradient, which then gets normalized by $\sqrt{\hat{v}_t}$ — and for parameters with large gradient variance, the regularization is effectively erased. Decoupled weight decay applies $\theta \rightarrow (1 - \eta\lambda)\theta$ as a *separate* step, untouched by Adam’s normalization. The result: actual regularization, lower test loss, more stable training. Every modern LLM uses AdamW.

Lion (Chen et al. 2023) and Sophia (Liu et al. 2023) have been proposed as cheaper alternatives — Lion uses sign-of-momentum updates, Sophia incorporates a diagonal Hessian estimate. Both have been shown to compete with AdamW at moderate scales. Whether they hold up at 10^{24} FLOPs is an open empirical question; AdamW remains the production default.

13.7 Learning-rate schedules

Two standard pieces:

- **Warmup.** Linearly ramp η from 0 to $\eta_{\max}$ over the first 0.5–2% of steps. Without warmup, the early gradients (from a near-randomly-initialized network) destabilize Adam’s running statistics: v_t underestimates the true second moment for several steps, causing oversized updates that diverge. Warmup lets v_t accumulate a representative estimate before η becomes large. Empirically, warmup is the difference between “trained” and “diverged” for large Transformers.
- **Cosine decay.** Decay η from $\eta_{\max}$ to a small floor (often $0.1\eta_{\max}$ or 0) on a half-cosine. Empirically robust across scales. Linear decay also works; constant-then-step-decay performs worse.

Why cosine specifically? It is partly aesthetic, but it has the property that the rate of decay is gentle near the start and end, with the steepest decay in the middle. Empirically this matches the regime where fine-grained adjustments matter more than coarse ones during the late stage of training. Recent work (Chinchilla schedules, μ P-aware schedules, the “WSD” warmup-stable-decay schedule used by MiniCPM and others) revisits the choice; cosine remains the broad default.

A subtler issue is *peak learning rate as a function of scale*. Without μ P, the optimal $\eta_{\max}$ tends to shrink as model width grows, requiring re-tuning per scale. With μ P, $\eta_{\max}$ is roughly width-invariant and can be transferred from small proxies.

13.8 Initialization for residual stacks

The classical Glorot/He initializations target unit variance through one layer. In a Transformer of L residual blocks, the residual stream’s variance grows roughly linearly in L if every block is initialized to contribute order-1 variance. By layer 50, the stream has diverged in magnitude, and downstream LayerNorms must work harder to renormalize.

The fix: scale the residual branch’s weights so that each block contributes $\mathcal{O}(1/L)$ to the stream’s variance. Common variants:

- **GPT-2’s scaling.** Scale the output projection of each residual sublayer by $1/\sqrt{2L}$ at initialization. The factor of 2 accounts for the two residual writes per block (MHA and MLP).
- **DeepNet** (Wang et al. 2022). A pre-norm variant that supports thousands of layers via carefully chosen residual gain coefficients.
- **μ P-derived rules.** Scale the residual path by $1/\sqrt{L}$ in conjunction with width scaling so that depth and width both behave under principled limits.

- **ReZero / Skiplinit.** Initialize the residual branch’s gain to zero; the network starts as the identity and is gradually filled in. Stable at extreme depth, slightly slower in early training.

The general principle: *the residual stream is a sum of L contributions; for the sum to remain bounded as L grows, each contribution must shrink with L .* Architectures that ignore this rule fail to train at depth — and the failure is often silent, manifesting as poor convergence rather than NaNs.

13.9 μ P and width-invariant tuning

μ P (Yang & Hu 2021; Yang et al. 2022 for transfer recipes) deserves a paragraph on its own. The core insight: there is a unique parameterization in which, in the infinite-width limit, every parameter’s update at every step is $\mathcal{O}(1)$ in the right sense — neither vanishing nor diverging — so that the network has well-defined, non-trivial learning dynamics in that limit.

The practical recipe:

- Output projection scaled by $1/d_{\text{out}}$ at the readout.
- Hidden weight initialization with variance $1/d_{\text{in}}$.
- Per-layer learning rate adjustments by width-dependent factors (typically the readout learns at rate η/d , hidden weights at rate η).
- Attention logits scaled by $1/d_k$ instead of $1/\sqrt{d_k}$ at infinite width (though the $1/\sqrt{d_k}$ choice is common in finite-width practice).

The payoff: hyperparameter tuning at $d = 256$ in a few thousand GPU-hours yields learning rate, weight decay, and warmup choices that transfer to $d = 8192$ at hundreds of millions of dollars of compute. The cost: the parameterization is non-trivial to retrofit, and many open codebases haven’t fully adopted it.

The architectural lesson: hyperparameters are not opaque “tuning knobs”; they are derived quantities of how you parameterize the network. A good parameterization makes the right choices nearly automatic. A bad one makes every scale-up a fresh adventure in stability tuning.

Chapter 14. Mixture-of-Experts

14.1 Sparsity as a scaling axis

Dense Transformers spend FLOPs proportional to active parameters. Mixture-of-experts (MoE) decouples the two: the model has many parameters; each token activates only a few.

A standard MoE layer replaces the MLP with E expert MLPs and a router. Given input x , the router produces logits $g(x) \in \mathbb{R}^E$, picks the top- k experts (typically $k = 2$), and outputs

$$\text{MoE}(x) = \sum_{e \in \text{TopK}(g(x), k)} \frac{\exp g(x)_e}{\sum_{e' \in \text{TopK}} \exp g(x)_{e'}} \cdot \text{Expert}_e(x).$$

Capacity scales with E ; FLOPs scale with k . In practice, a 64-expert top-2 MoE has $\sim 30\times$ the parameters of a dense baseline at the same FLOPs.

14.2 Load balancing

Naïve MoE training collapses: a few experts get all the tokens; the rest die. The standard fix is an auxiliary load-balancing loss (Shazeer et al., 2017):

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_e f_e \cdot p_e,$$

where f_e is the fraction of tokens routed to expert e and p_e is the average router probability for e . Minimizing this pushes $f_e \cdot p_e$ toward the uniform value $1/E^2$.

Other tricks: capacity factors (cap tokens per expert), token dropping at full experts, expert dropout. Modern MoEs (Switch, GShard, Mixtral, DeepSeek-V2/V3) tune all of these.

14.3 Why MoE scales capacity but not simplicity

What MoE buys:

- Parameter count without proportional FLOPs.
- Specialization: experts often (mildly) specialize to languages, topics, code styles.
- Better loss at fixed compute, in many regimes.

What MoE costs:

- **Memory.** All experts must live somewhere. Inference memory does not shrink.
- **Communication.** Across-device routing is bandwidth-heavy; MoE training requires careful all-to-all collectives.
- **Routing instability.** Routers can collapse, oscillate, or develop pathological specializations.
- **Inference complexity.** Serving an MoE efficiently is substantially harder than serving a dense model — a real engineering tax.

The research jury is still out. DeepSeek-V3 (671B total, 37B active) is one of the strongest open models of its era; many production systems still prefer dense for serving simplicity. MoE is a tool, not a destination.

14.4 Token-choice vs expert-choice routing

The standard MoE router is *token-choice*: each token picks the top- k experts. This guarantees that every token is processed but does not guarantee load balance — popular experts can be flooded while others go dark.

Expert-choice routing (Zhou et al. 2022) inverts the assignment: each expert picks the top- k tokens for its capacity slot. Now load is balanced *by construction* — every expert sees exactly its capacity. The cost: some tokens may be selected by zero experts and skipped (or passed through a residual unmodified). Empirically, expert-choice trains faster and matches or beats token-choice on quality, particularly at scale.

A third option, **soft routing** (Soft MoE, Puigcerver et al. 2023), gives every token a softmax-weighted blend of all experts. This eliminates the discrete routing decision (good for gradients) but loses sparsity (every expert runs on every token). Soft MoE works in vision but has been less studied for language.

A pragmatic compromise gaining traction: **shared experts plus routed experts**. DeepSeek-V2/V3 keep one or two “always-on” experts that every token sees, plus a larger pool of routed experts among which the router picks top- k . The shared experts catch generic computations; the routed experts specialize. Routing collapse is rarer because the shared expert provides a baseline.

Variant	Compute scaling	Load balance	Backprop friendliness	Production adoption
Token-choice	Sparse, k experts/token	Needs aux loss	Discrete, biased ST	Switch, GShard, Mixtral
Expert-choice	Sparse, capacity-bounded	By construction	Discrete	Some Google internal
Soft MoE	Dense over E experts	By construction	Smooth	Vision; rare for language
Hash routing	Sparse	By construction	No gradient on router	Research
Shared + routed	Hybrid	By construction	Discrete on routed	DeepSeek-V2/V3

There is no single winner; the choice is a constraint reallocation between sparsity, balance, and gradient quality.

14.5 Failure modes in detail

MoEs fail in characteristic ways. A handful of the most important:

- **Router collapse.** All tokens go to a single expert; the rest receive no gradient and slowly drift into dead weight. Auxiliary losses, capacity factors, and noise injection all combat this. A stable router is the precondition for any other gain.
- **Brittle specialization.** Specialization is desirable; *brittle* specialization is not. If exactly one expert handles, say, code, and that expert is unavailable (due to capacity overflow or device failure), code performance collapses. Soft fallbacks (shared experts, top-2 with high overlap) mitigate this.
- **Train/serve skew.** Training often uses generous capacity factors and tolerates dropped tokens; serving uses tight capacity factors for latency. The router learned at training may not behave well at serving, especially for adversarial inputs that route preferentially to low-capacity experts.
- **Routing oscillation.** During training, the router can flip between assignment regimes, slowing convergence. Step-size and noise-level tuning matter more than for dense models.
- **Gradient bias.** Top- k is non-differentiable. The straight-through estimator is biased; alternatives (Gumbel-softmax, REINFORCE with variance reduction) trade variance for bias. The bias is small in well-tuned setups but it is real and accumulates over long training runs.
- **Token dropping at inference.** If incoming traffic patterns differ from training (e.g., a customer’s prompts are all about one topic, hammering one expert), capacity overflow causes silent quality loss. Production systems need careful overflow handling, often with fallback paths.

The honest summary: training a stable, fast MoE requires substantially more engineering than training a dense Transformer of similar capability. The capability-per-FLOP gain has to overcome that engineering tax — and at small-to-medium scales, it often doesn’t.

14.6 Engineering: all-to-all and expert parallelism

Routing is the easy part; *executing* the routed work efficiently across hundreds or thousands of devices is hard. The standard pattern, **expert parallelism**, places different experts on different devices. After the router decides which expert each token visits, an *all-to-all collective* shuffles tokens across devices so each device receives the tokens assigned to its experts; the experts run their MLPs locally; another all-to-all returns outputs to the originating devices for the residual write.

The cost is bandwidth-bound. All-to-all moves $\mathcal{O}(\text{tokens} \cdot d)$ bytes per layer per pass, twice (once forward, once backward). On modern interconnects (NVLink, NVSwitch, InfiniBand HDR/NDR), this is feasible at the scale of single training runs. On commodity Ethernet, MoE is infeasible; the all-to-all serializes and dominates wall-clock time. The architecture's *physical* deployment is part of the architectural choice.

Standard mitigations:

- **Local experts first.** If $k = 2$ and one of the two chosen experts lives on the local device, route there first; only one all-to-all is needed instead of two.
- **Topology-aware routing.** Bias the router slightly toward experts on the same node or same NUMA domain.
- **Compute-communication overlap.** While one micro-batch's all-to-all is in flight, another micro-batch's experts run.
- **Token grouping.** Sort tokens by their target expert so that the all-to-all's send buffer is contiguous; reduces serialization overhead in the collective.

The lesson: **MoE is a distributed-systems architecture, not just a model architecture.** Treating it as the latter — picking E, k without thinking about all-to-all — is how MoE projects fail in production despite training-loss curves that look great.

Chapter 15. Long-Context Architectures

15.1 The T^2 wall

Attention costs $\mathcal{O}(T^2 d)$ in compute and $\mathcal{O}(T^2)$ in memory (for the attention matrix during training). At $T = 1,024$, this is a footnote. At $T = 128,000$, it dominates. Doubling T quadruples cost.

Long context matters: code, documents, books, conversations, retrieval-augmented prompts. The community has explored four broad routes around the T^2 wall.

15.2 Sparse attention

Each position attends only to a structured subset:

- **Local windows** (Longformer, BigBird): each token attends to a window of w neighbors.
- **Strided / dilated** patterns: cover long range with logarithmic depth.
- **Global tokens:** a few tokens attend to all and are attended by all (memory tokens).

Cost drops to $\mathcal{O}(Tw)$ or $\mathcal{O}(T\sqrt{T})$. Quality: usually OK for tasks dominated by local structure; can fail on truly global dependencies.

15.3 Linear attention

Replace softmax with a kernel:

$$\text{softmax}(QK^\top)V \rightarrow \phi(Q)(\phi(K)^\top V),$$

with ϕ chosen so that the right-hand side is computable in $\mathcal{O}(Td^2)$ rather than $\mathcal{O}(T^2d)$. Performer, Linear Transformer, Linformer, and others sit in this family.

The catch: linear attention loses the *dynamic, content-addressed* property of softmax attention. It approximates the kernel; it does not approximate the *router*. Quality often suffers, and the gap is hard to close without giving back the speedup.

15.4 KV-cache constraints

At inference, autoregressive decoding stores the keys and values of every previous position. KV cache size is

$$\text{KV bytes} = 2 \cdot L \cdot T \cdot h_{\text{kv}} \cdot d_k \cdot \text{bytes/float}.$$

For LLaMA-3-70B at $T = 128k$ in fp16, this is dozens of GB *per request*. Solutions stack:

- **GQA / MQA.** Reduce h_{kv} .
- **Quantization.** 4-bit or 2-bit KV (with careful scaling).
- **Cache eviction.** Drop low-attention positions. StreamingLLM keeps a rolling window plus “attention sinks.”
- **Cache compression.** Compress old positions into summary slots.

These are systems decisions, not architectural ones, but they are what makes long context practical.

15.5 FlashAttention

Dao et al. (2022) observed: the attention computation is **memory-bandwidth-bound**, not compute-bound. The naïve implementation materializes the full $T \times T$ attention matrix in HBM. FlashAttention reorders the computation to keep tiles in SRAM, computing softmax in an online fashion and never materializing the full matrix.

The key identity is the **online softmax**:

$$\text{softmax}(z_1, \dots, z_n) = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}},$$

where the running max m and running sum can be updated incrementally as new tiles arrive. Combined with a careful tiling of Q, K, V blocks, FlashAttention reduces HBM reads/writes from $\mathcal{O}(T^2)$ to $\mathcal{O}(T^2/B)$ for block size B .

The result: $2\text{--}4\times$ wall-clock speedup, $10\text{--}20\times$ memory savings, and exactly the same numerics as standard attention. FlashAttention-2 and FlashAttention-3 push this further with better warp scheduling and async copies. **FlashAttention is the single most important systems contribution to LLM training in the last five years.**

The lesson: **the relevant cost is not always FLOPs; it is bytes moved.** Roofline thinking matters as much as algorithmic complexity.

15.6 The online softmax derivation, in full

FlashAttention's correctness rests on an old identity. To compute softmax in tiles without storing the full vector, maintain a running maximum m and a running normalizer $\ell = \sum_j e^{z_j - m}$.

Suppose we have processed a tile and have running quantities $(m^{(1)}, \ell^{(1)}, o^{(1)})$, where

$$o^{(1)} = \sum_{j \in \text{tile}_1} \frac{e^{z_j - m^{(1)}}}{\ell^{(1)}} v_j$$

is the attention-weighted value over tile 1. A new tile arrives with raw logits $z^{(2)}$ and values $v^{(2)}$. Compute its local max $m_{\text{loc}}^{(2)} = \max_{j \in \text{tile}_2} z_j$ and local sum $\ell_{\text{loc}}^{(2)} = \sum_{j \in \text{tile}_2} e^{z_j - m_{\text{loc}}^{(2)}}$.

The merged max is $m^{(\text{new})} = \max(m^{(1)}, m_{\text{loc}}^{(2)})$. The merged normalizer is

$$\ell^{(\text{new})} = e^{m^{(1)} - m^{(\text{new})}} \ell^{(1)} + e^{m_{\text{loc}}^{(2)} - m^{(\text{new})}} \ell_{\text{loc}}^{(2)}.$$

The merged output is

$$o^{(\text{new})} = \frac{e^{m^{(1)} - m^{(\text{new})}} \ell^{(1)}}{\ell^{(\text{new})}} \cdot o^{(1)} + \frac{e^{m_{\text{loc}}^{(2)} - m^{(\text{new})}} \ell_{\text{loc}}^{(2)}}{\ell^{(\text{new})}} \cdot \sum_{j \in \text{tile}_2} e^{z_j - m_{\text{loc}}^{(2)}} v_j.$$

The arithmetic is *bit-equivalent* to the one-shot softmax, modulo floating-point rounding (which empirically matches naïve attention to within 10^{-3} in fp16). The IO complexity, by contrast, is changed dramatically: the full $T \times T$ attention matrix never materializes in HBM. For block size B , the matrix appears only in $B \times B$ tiles in SRAM.

The HBM read/write count goes from $\Theta(T^2)$ in naïve attention to $\Theta(T^2 d / M)$ where M is SRAM capacity. The dominant data movement is now the $\Theta(Td)$ inputs and outputs, with attention scores computed on chip and discarded. For typical $d \approx 128$, $T \approx 16k$, $M \approx 100$ KB per SM, the HBM savings are $10\text{--}20\times$.

This is the rare case where a careful re-implementation yields not a $2\times$ but an order-of-magnitude memory reduction *with identical numerics*. It is a textbook example of *roofline-aware* algorithm design: identify the bandwidth bottleneck, restructure the computation to avoid it, accept slightly more arithmetic in exchange.

15.7 KV-cache compression and paged attention

The KV cache dominates inference memory at long context. Several compression techniques have emerged:

- **Quantization.** Store K, V in 4-bit or 2-bit with per-channel scales. Quality loss is small (often $< 1\%$ perplexity hit) if scales are chosen carefully; quality cliffs are sharp below 2 bits.
- **Low-rank or sparse projections.** Project K, V to a smaller dimension at the cost of some quality. KIVI, DenseKV, and others fall in this family.
- **Selective eviction.** Drop tokens with low historical attention weight (StreamingLLM, H₂O, SnapKV). StreamingLLM keeps a constant cache size by sliding-window plus *attention sinks* — initial tokens that disproportionately attract attention because the softmax cannot output zero everywhere.
- **PagedAttention** (Kwon et al. 2023). The cache is allocated as fixed-size pages with an indirection table per request. This eliminates fragmentation and enables efficient prefix sharing across requests with common system prompts. vLLM’s implementation drives most modern open-source serving stacks.
- **Cross-request prefix caching.** When many requests share a long system prompt, compute its KV cache once and reuse it. The savings are linear in concurrent requests.

The shift in mental model: the KV cache is an *operating system problem*, not an algorithm problem. Once that framing is accepted, paged attention follows almost immediately from the last fifty years of virtual memory’s lessons.

15.8 Length generalization: ALiBi, position interpolation, NTK-aware, YaRN

Trained on contexts up to T_{train} , can a model handle $T > T_{\text{train}}$? Often poorly, and the position encoding is the usual culprit.

A taxonomy of fixes:

- **ALiBi** (Press et al. 2021). Adds a linear bias $-m \cdot |i - j|$ to attention logits, with m a per-head constant. Heads can extrapolate to several times the training length because the bias is well-defined for any distance. ALiBi-trained models extrapolate cleanly but lose some short-context performance; it is the encoding of choice in some recent open models (e.g., Replit Code) but not most.
- **Position interpolation** (Chen et al. 2023). For RoPE-trained models, scale the position index by $T_{\text{train}}/T_{\text{eval}}$ at inference. This effectively rescales RoPE’s rotational frequencies, preserving training-time geometry. Works for moderate extrapolation factors (2–4×) and is the cheapest length extension technique.
- **NTK-aware scaling** (LocalLLaMA community 2023; later formalized). Modify RoPE’s base frequency θ_{base} to push high-frequency components to longer wavelengths. Less aggressive than uniform interpolation; better at preserving short-range fidelity at the cost of more complex tuning.
- **YaRN** (Peng et al. 2023). A combination: interpolate low-frequency components heavily, leave high-frequency components alone, with a smooth transition between regimes. The current best off-the-shelf method for RoPE extrapolation up to roughly 16× the training length without quality collapse.
- **Continued pretraining at long context.** The cleanest fix when compute is available: train on long sequences. LLaMA-3, Qwen-2, and others use this, often with progressive length curricula (start at 2k, extend to 8k, then 32k, then 128k). The cost is the long-sequence training itself, which is exactly what we were trying to avoid.

The lesson: positional encoding is not just a way to break permutation symmetry; it determines length generalization. Architectures that confuse “absolute” with “relative” pay for it at long context, and the price is usually paid as a sudden quality cliff just past T_{train} rather than as a graceful degradation.

Chapter 16. State-Space Models

16.1 The recurrent revival

If attention’s T^2 is the binding constraint at long context, perhaps recurrence — which is $\mathcal{O}(T)$ — is worth reconsidering, this time with modern math and modern accelerators in mind.

The state-space model (SSM) starts from continuous-time dynamics:

$$\dot{\mathbf{h}}(t) = A\mathbf{h}(t) + Bu(t), \quad y(t) = C\mathbf{h}(t) + Du(t).$$

Discretized by zero-order hold or bilinear transforms, this becomes a linear recurrence:

$$\mathbf{h}_t = \bar{A}\mathbf{h}_{t-1} + \bar{B}u_t, \quad y_t = C\mathbf{h}_t + Du_t.$$

For fixed $\bar{A}, \bar{B}, C, D$, this recurrence can also be written as a *convolution*:

$$y_t = \sum_{k=0}^t (C\bar{A}^k \bar{B})u_{t-k} = (\bar{K} * u)_t,$$

so training can be done with FFT-based convolution in $\mathcal{O}(T \log T)$, while inference uses the recurrent form in $\mathcal{O}(T)$. Best of both worlds — *if* you can choose A wisely.

16.2 S4

S4 (Gu, Goel, Ré 2022) achieved this with a clever HiPPO parameterization of A that gives long-range memory while keeping the convolutional kernel computable cheaply via diagonal-plus-low-rank structure. S4 dominated long-range benchmarks (Long-Range Arena), often by huge margins.

16.3 Mamba: selective recurrence

S4’s $\bar{A}, \bar{B}$ are *input-independent* — they form a linear time-invariant system. That makes them efficient but also weakens content-based selectivity that attention has natively. Mamba (Gu & Dao 2023) made $\bar{B}$ and $\bar{C}$ functions of the input:

$$\bar{B}_t = f_B(u_t), \quad \bar{C}_t = f_C(u_t).$$

This breaks the convolutional speedup — but with careful CUDA kernels (parallel scan), it remains $\mathcal{O}(T)$ in practice. Mamba achieves Transformer-level quality with linear-time inference; a hybrid Mamba/attention model can match dense Transformers at long contexts with much smaller KV caches.

16.4 Trade-offs vs attention

Property	Attention	SSM (S4 / Mamba)
Training compute	$\mathcal{O}(T^2)$	$\mathcal{O}(T \log T)$ or $\mathcal{O}(T)$
Inference compute / token	$\mathcal{O}(T)$	$\mathcal{O}(1)$
KV / state memory	$\mathcal{O}(T)$	$\mathcal{O}(1)$ (fixed state)
Content-based routing	Native	Limited; selectivity helps
Path length to history	1	Implicit, decays
Hardware fit	Excellent	Good with custom kernels

The honest summary: SSMs are not strictly better than attention; they are differently constrained. They handle million-token contexts more gracefully but lose some sharp content-addressed reasoning. Hybrid stacks — a few attention layers interleaved with many SSM layers — are emerging as a practical middle ground.

The lesson: **recurrence is back, but only because we learned how to make it parallel**. The original sin of vanilla RNNs was sequential training; SSMs erase that sin via convolutional reformulation and parallel scans.

16.5 HiPPO: why this A matrix?

S4's secret sauce is the choice of A . Random A does not work; carefully chosen A does. The right framework is **HiPPO** (Gu et al. 2020).

The HiPPO question: given a continuous signal $u(t)$, what dynamical system $\dot{\mathbf{h}}(t) = A\mathbf{h}(t) + Bu(t)$ produces a hidden state $\mathbf{h}(t)$ whose components are coefficients of a *continuous-time approximation* of the signal's history in a chosen orthogonal-polynomial basis?

For the Legendre polynomials with a sliding-scaled measure (HiPPO-LegS), the answer is

$$A_{ij} = \begin{cases} -(2i+1)^{1/2}(2j+1)^{1/2}, & i > j \\ -(i+1), & i = j \\ 0, & i < j, \end{cases} \quad B_i = (2i+1)^{1/2}.$$

With this A , the hidden state $\mathbf{h}(t)$ at any time *is* a compressed, lossy representation of the input's full past in a Legendre basis, with the approximation refining as the state dimension N grows. Other bases (Laguerre, Fourier) give different A matrices with different memory profiles (exponentially decaying, oscillatory, etc.).

S4's algorithmic contribution is the observation that this same A — together with carefully chosen complex eigenvalues — has a *diagonal-plus-low-rank* structure. The convolutional kernel $\bar{K}_t = C\bar{A}^t\bar{B}$ can therefore be computed in $\mathcal{O}(N \log N)$ via Cauchy / DFT machinery. The matrix that is *mathematically motivated* by signal-approximation theory is also *computationally efficient*. This coincidence is what makes S4 work — and the failure of naïve random A shows the brittleness underneath.

Mamba abandons the convolutional kernel when it makes $\bar{B}, \bar{C}$ input-dependent (so the system is no longer linear time-invariant), but it keeps the diagonal A — the long-memory backbone — and adds selectivity on top. The HiPPO heritage shows up in initialization: Mamba initializes its diagonal A at HiPPO-derived values, then lets training adjust them.

The lesson: the difference between “an SSM” and “an SSM that works” is the spectral structure of A . Random initialization fails; HiPPO-derived initialization succeeds. The literature underestimates how brittle this choice is — and how much of the SSM’s empirical success rides on a single carefully constructed matrix family.

16.6 Parallel scan: how Mamba stays $\mathcal{O}(T)$ in practice

A naïve recurrent computation $\mathbf{h}_t = \bar{A}_t \mathbf{h}_{t-1} + \bar{B}_t u_t$ is $\mathcal{O}(T)$ wall-clock because each step depends on the previous. On a GPU, this is exactly the property that made vanilla RNNs slow. How does Mamba achieve $\mathcal{O}(T)$ training without sequential bottleneck?

Associative scan. The recurrence is a sequence of affine operations $f_t : \mathbf{h} \mapsto \bar{A}_t \mathbf{h} + \bar{B}_t u_t$, each parameterized by a pair $(\bar{A}_t, c_t)$ where $c_t = \bar{B}_t u_t$. Composition of two such operations is associative:

$$(f_{t+1} \circ f_t)(\mathbf{h}) = \bar{A}_{t+1}(\bar{A}_t \mathbf{h} + c_t) + c_{t+1} = (\bar{A}_{t+1} \bar{A}_t) \mathbf{h} + (\bar{A}_{t+1} c_t + c_{t+1}).$$

So the composition has the same form, with new pair $(\bar{A}_{t+1} \bar{A}_t, \bar{A}_{t+1} c_t + c_{t+1})$. Associativity means we can compute partial compositions in a binary tree:

- Level 0: T leaves, one per timestep.
- Level 1: $T/2$ pairwise compositions in parallel.
- Level 2: $T/4$ four-element compositions in parallel.
- ...
- Level $\log T$: a single global composition.

Total work: $\Theta(T)$. Critical-path depth: $\Theta(\log T)$. If a GPU can fit T work units, the wall-clock time is $\Theta(\log T)$ steps of parallel composition rather than T sequential steps.

Mamba’s CUDA implementation uses this scan, fused with the kernel that produces $\bar{A}_t, \bar{B}_t, \bar{C}_t$ from the input, and it avoids materializing the hidden states explicitly in HBM — they live transiently in registers and SRAM. The result is a GPU-saturated training loop that scales gracefully to long sequences and outperforms attention’s T^2 wall by an asymptotic factor.

The lesson: *recurrence and parallelism are not antonyms*. They are antonyms only if the recurrence is non-associative or if you implement it naïvely. Choose your operator carefully — affine recurrences, max-plus semirings, certain matrix recurrences all qualify — and recurrence runs as fast as attention while scaling far better in T .

Chapter 17. Retrieval-Augmented Generation

17.1 Parametric vs non-parametric memory

Knowledge stored in model weights is *parametric* memory: cheap to query (forward pass), expensive to update (retraining), costly to scale (parameter count, training tokens). Knowledge stored in an external corpus is *non-parametric* memory: cheap to update (just edit the corpus), cheap to scale (terabytes), but requires retrieval to use.

Retrieval-augmented generation (RAG) is the architectural pattern that combines both: a retriever fetches relevant documents; a generator conditions on them.

17.2 The basic pipeline

1. **Index** a corpus by computing dense embeddings (e.g., BGE, E5, OpenAI ada) and storing them in a vector index (FAISS, ScaNN, HNSW).
2. **Retrieve** the top- k documents for a query embedding.
3. **Generate** with the LLM conditioned on [retrieved docs; query].

This is not a new architecture so much as a *system architecture*. The LLM itself is unchanged. But the implications for what the LLM must store internally are profound.

17.3 Architectural vs system-level memory

There is a real research question buried here: should knowledge live in weights or in a database?

- **In weights:** fast, integrated reasoning over knowledge. But updating means retraining; auditing is impossible; size scales with parameter count.
- **In retrieval:** auditable, updatable, scalable. But the LLM must learn to *use* retrieved context faithfully; retrieval errors compound.

Hybrid is winning: train the model to be *good at using* retrieved context (instruction tuning, long-context training), and offload most factual knowledge to retrieval. The cost is that *reasoning over* retrieved context is still imperfect — distractor passages can derail otherwise capable models.

The constraint reallocation: trade parametric capacity (size, freshness) for non-parametric capacity (corpora, indexes), at the cost of retrieval fragility.

17.4 Variations on a theme

- **REALM, RAG (Lewis 2020), Atlas:** end-to-end trained retriever + generator.
- **Re-rankers:** a cross-encoder reorders the top- k from the bi-encoder retriever.
- **Tool-augmented LLMs:** retrieval is one of many tools (search, calculators, code execution, APIs).
- **Long-context-as-RAG:** sometimes you can just stuff the corpus into the prompt. With 1M-token contexts, the line between RAG and “just put it in the prompt” blurs.

The lesson: **memory is a place, not a property**. Good architectures decide deliberately what to keep where.

17.5 Chunking and embedding choices

The first real engineering decision in RAG is *chunking*: how do you split a corpus into retrievable units?

- **Fixed-size chunks** (e.g., 512 tokens). Simple; loses semantic boundaries (cuts mid-paragraph or mid-sentence).
- **Sentence- or paragraph-based**. Respects natural boundaries but produces chunks of widely varying lengths, complicating embedding statistics.
- **Recursive splitting**. Try splitting on heading boundaries first; fall back to paragraph; then sentence; then character. LangChain's `RecursiveCharacterTextSplitter` is the canonical implementation of this idea.
- **Semantic chunking**. Detect topic boundaries via embedding-similarity drops; segment between them. More principled, more expensive.
- **Hierarchical / parent-child**. Index small chunks for retrieval, but pass larger surrounding context (the parent paragraph or section) to the LLM. A widely-used pattern: small for matching, large for grounding.

The second decision is the *embedding model*. Notable families:

- **OpenAI text-embedding-3-small/large**. Strong general-purpose; commercial.
- **Cohere embed-v3**. Strong; commercial; multilingual.
- **BGE (BAAI), E5 (Microsoft)**. Open-weights; competitive with commercial. BGE-M3 covers dense, sparse, and multi-vector modes from one model.
- **MiniLM, MPNet**. Smaller, older, still useful for cost-sensitive deployments.
- **Domain-specific**. Code-aware (CodeBERT, jina-embeddings-v2-code), biomedical (BioBERT), legal, finance.

Embedding choice interacts with chunking: chunks that exceed the embedder's maximum input length get truncated silently, producing degraded representations. Production RAG stacks routinely run a configuration matrix of (chunk size × embedder × index type) over a held-out evaluation set before settling on a deployment.

17.6 Re-ranking with cross-encoders

Bi-encoder retrieval (separate encoders for query and document, dot-product to compare) is fast — embeddings are precomputed — but loses interaction information. Cross-encoders pass the (query, document) pair jointly through a small Transformer and score them holistically; far more accurate, far slower.

The standard pattern: **two-stage retrieval**.

1. **Bi-encoder pass**. Retrieve top- K_1 (e.g., 100) candidates by dot-product over embeddings. Cheap; sub-second over millions of documents with HNSW or IVF indices.
2. **Cross-encoder pass**. Score each of the K_1 candidates jointly with the query; keep the top- K_2 (e.g., 5–10). Expensive per item ($\sim$ ms each on GPU), but only K_1 items, so total cost is bounded.

Strong cross-encoders (BGE-reranker, Cohere Rerank, mxbai-rerank) reorder bi-encoder results to recover precision lost in the embedding bottleneck. Two-stage retrieval consistently outperforms either stage alone.

17.7 Query rewriting, HyDE, and multi-vector

Naive RAG retrieves documents that match the *query's surface form*. But users ask vague, conversational questions; documents are written in declarative prose. The vocabulary mismatch is the dominant failure mode of cheap RAG systems.

Mitigations:

- **Query rewriting.** Use the LLM to rewrite the query into a search-engine-friendly form before retrieval. Cheap LLMs handle this well; the rewriting becomes an extra step in the pipeline.
- **HyDE** (Hypothetical Document Embeddings; Gao et al. 2022). Have the LLM *hallucinate* a plausible answer to the query, then embed and retrieve real documents that match the hallucination. Surprising and effective: the hallucinated answer is in the language of documents, not queries.
- **Query expansion.** Run several semantically related queries (synonyms, paraphrases, sub-questions) and union the retrieved sets.
- **Multi-vector retrieval (ColBERT).** Index *every token* of every document, not just a single document embedding. At query time, score by maxsim: each query token finds its best-matching document token; sum across query tokens. Heavy but quality-dominant.

Each of these techniques pays compute or storage to recover precision. The right choice is determined by the *failure mode* in your retrieval evaluation: vocabulary mismatch (HyDE), under-ranking of relevant docs (re-ranking), missing facets (query expansion), token-level mismatch (multi-vector).

17.8 Failure modes of RAG

RAG fails in characteristic ways. The most important:

- **Distractor hallucination.** A retrieved document is mostly relevant but contains a wrong fact; the LLM faithfully reproduces the wrong fact. The user trusts the citation; the citation supports the error. This is the dominant production failure mode.
- **Citation drift.** The LLM produces an answer based on its parametric knowledge, then *attaches* a citation that is only loosely related. Looks credible, isn't. A faithful generation prompt and post-hoc citation verification help.
- **Context contamination.** Retrieving too many chunks (e.g., $K = 50$) buries the relevant ones in noise. The LLM's effective attention is finite; performance can *degrade* with more context. Most production stacks stop at $K = 3-8$.
- **The lost-in-the-middle effect** (Liu et al. 2023). Information placed in the middle of a long context is recalled less reliably than information at the start or end. Important for ordering: put the most-relevant chunk first or last.
- **Stale or contradictory corpora.** Multiple documents say different things; the LLM averages or arbitrarily picks. Curating the corpus matters as much as the retriever.
- **Adversarial documents.** A maliciously crafted document with prompt-injection content (e.g., "ignore previous instructions") in the retrieved chunk can hijack the LLM. A real and growing attack surface for production RAG.

The architectural lesson: *RAG is not a magic factuality button*. It is a pipeline with at least four failure points (chunking, embedding, retrieval, generation), each of which can silently degrade quality. Production RAG is a measurement-and-iteration discipline as much as it is an architecture.

Chapter 18. Multimodal Architectures

18.1 Why Transformers generalize across modalities

A Transformer is content-agnostic: it processes a sequence of vectors. If you can turn images, audio, video, or proteins into sequences of vectors, the same architecture works. This explains the empirical fact that “everything-to-tokens” works.

18.2 Tokenization across modalities

- **Images.** Patch-based: cut into 16×16 patches, linearly embed each (ViT, Dosovitskiy 2020). Or VQ-based: encode with a VQ-VAE/VQ-GAN into discrete codes (used by Parti, Chameleon).
- **Audio.** Mel-spectrogram patches; or discrete acoustic tokens via SoundStream/Encodec; or wav2vec features.
- **Video.** Spatiotemporal patches (often $16 \times 16 \times t$); or per-frame tokens with temporal attention.
- **Proteins, molecules, code, DNA.** Already discrete or easily discretized.

The act of tokenization is itself a major architectural decision. It bakes in inductive bias about what counts as an “atomic unit” of the modality.

18.3 Fusion strategies

Three major patterns:

1. **Early fusion (single stream).** All modalities are tokenized and concatenated; one big Transformer processes them. Simplest; works well at scale (GPT-4V, Chameleon, Gemini Pro).
2. **Late fusion (separate encoders + cross-attention).** Vision encoder produces tokens; LLM cross-attends to them. Used by Flamingo, BLIP-2, LLaVA.
3. **Adapter/projector approaches.** A frozen vision encoder’s output is projected into the LLM’s embedding space via a small MLP. LLaVA’s recipe; cheap and effective for vision-language.

Trade-offs: early fusion is uniform but expensive; late fusion is modular but harder to train end-to-end; adapter-style is cheap and scrappy.

18.4 Why this works

Two empirical regularities:

- **The Transformer is a generic sequence-to-sequence learner.** Given enough data and a sane tokenization, modality detail is mostly absorbed into the input embeddings.
- **Pretraining on language transfers.** Language-only pretraining gives strong reasoning, instruction-following, and abstraction; bolting on vision/audio is far cheaper than starting from scratch.

The architectural lesson: **modalities are interfaces, not architectures.** The serious work is at the tokenizer and the alignment data. The Transformer beneath is ordinary.

18.5 Vision encoders: ViT, CLIP, DINOv2, SigLIP

Most multimodal LLMs use a pretrained vision encoder rather than training visual representations from scratch. The choice of encoder is consequential.

- **ViT (Dosovitskiy 2020)**. The original; trained with supervised classification on ImageNet or with CLIP on web image-text pairs.
- **CLIP / OpenCLIP**. Contrastive image-text pretraining at web scale. Produces image features aligned with language; the alignment is the asset, not the classification accuracy.
- **DINOv2**. Self-supervised, trained without labels. Produces dense features that excel on segmentation, depth estimation, and other downstream tasks. Less aligned with language out of the box.
- **SigLIP** (Zhai et al. 2023). Pairwise sigmoid loss instead of softmax contrastive. Easier to scale to large batches; matches or beats CLIP at fixed compute. Used as the vision tower in PaliGemma, Idefics-3, and others.
- **InternVL / EVA-CLIP**. Larger CLIP-style encoders with stronger image features at higher resolution.

The choice depends on what the LLM needs to consume. For instruction-following and VQA, language-aligned features (CLIP/SigLIP) are usually preferred. For dense prediction, DINOv2 wins. Many production models combine multiple encoders or use a hybrid (e.g., InternVL-2 stitches a CLIP encoder and a high-resolution dense encoder; Cambrian-1 deliberately benchmarks across many).

Resolution matters more than people initially thought. Most vision encoders are pretrained at 224×224 or 336×336 . For OCR, document understanding, or fine detail, this is far too low. Recent multimodal models (LLaVA-NeXT, InternVL-2, Qwen2-VL) use *dynamic resolution*: tile the image into multiple crops, encode each, concatenate the resulting tokens. The token budget grows with image area, but quality follows. This is a clear example of constraint reallocation: trade context length for visual fidelity.

18.6 The projector problem

Once the vision encoder produces features (say, 729 patches of dimension 1024 from a 27×27 grid at 384 resolution), how do those features enter the LLM?

- **Linear projector** (LLaVA 1.5). A single linear layer maps $\mathbb{R}^{1024} \rightarrow \mathbb{R}^{4096}$. Cheap; sometimes too low-capacity.
- **MLP projector** (LLaVA-NeXT and most modern). Two-layer MLP with GeLU. Slightly more capacity; the standard.
- **Q-Former** (BLIP-2). A small Transformer with learned query tokens that *summarizes* the vision features into a fixed number of slots (typically 32). Fewer, richer tokens; more complex pipeline.
- **Resampler** (Flamingo). Learned latent queries cross-attend to vision features; output is fed via cross-attention into the LLM rather than as input tokens.
- **Native patches** (Chameleon, early-fusion models). Vision tokens are first-class token-IDs in the LLM's vocabulary; no projector at all.

The projector determines token economy. A long token output (729 tokens per image) eats into the LLM's context budget; a short one (32) constrains visual fidelity. Production multimodal LLMs have spent significant engineering on this trade-off — token compression schemes (pixel shuffle, token merging, learned downsampling) are an active research area.

18.7 Joint training dynamics

Training a Vision-Language Model (VLM) is not just “add vision data.” Several non-trivial dynamics arise:

- **Catastrophic interference.** If the vision data is added to the text mixture and the model is trained jointly, the LLM’s text capabilities can degrade, sometimes severely. Mitigations: freeze the LLM during early vision pretraining (LLaVA Stage 1), or unfreeze with a low learning rate later.
- **Modality balance.** The optimal ratio of text-only to vision-language batches is empirical. Too little text and the model loses language polish; too little vision and it cannot ground.
- **Resolution scheduling.** Train at low resolution first (cheaper) then high resolution, mirroring long-context curricula in pure text models.
- **Vision encoder freezing.** Freeze the encoder during the first phase to prevent it from drifting; unfreeze in a later phase to fine-tune to the LLM’s representations. The choice trades stability against alignment quality.

A standard recipe, distilled across LLaVA, BLIP-2, InternVL, and Qwen-VL:

1. *Stage 1 — alignment.* Train the projector only, on image-caption pairs. Cheap; aligns vision features to LLM input space.
2. *Stage 2 — instruction tuning.* Train the projector and LLM (often with a small learning rate) on visual instruction data.
3. *Stage 3 — full fine-tune (optional).* Unfreeze the vision encoder for a final low-LR phase.

Each stage has different optimal learning rates, batch sizes, and data mixes. The architecture is unchanged across stages; the *training* is what makes the model multimodal.

18.8 Discrete vs continuous tokens

A fundamental design choice for multimodal generation: are non-text outputs *generated* as tokens, or *decoded* by a separate model?

- **Continuous, separate decoder.** The LLM outputs a latent vector; an external diffusion model or VAE renders the image. Used by GPT-4V (originally), DALL-E 3 with GPT-4 prompting, Imagen-style pipelines.
- **Discrete, integrated.** The LLM emits image tokens from a learned VQ codebook (typically 8k–16k codes) that are decoded by a VQ-VAE. Chameleon, Parti, Emu-3 take this route. Image generation is *just next-token prediction* over a multi-modal vocabulary.
- **Hybrid.** Discrete planning tokens, continuous rendering. Some recent video models go this way to get best of both.

The discrete route is architecturally cleaner: the LLM does everything; modality differences are just vocabulary differences. The continuous route preserves perceptual quality (VQ codebooks lose detail) but requires two systems to maintain.

The trend: discrete is winning at the *interface* (text-image conversation, where modality fluidity matters more than peak fidelity) and continuous is winning at the *fidelity ceiling* (best-of-class image generation). Hybrids — discrete planning, continuous rendering — are likely to dominate the next generation.

18.9 Audio, video, and the temporal cost

Audio and video introduce a constraint that text and images do not: *time*. A 5-minute video at 30 FPS is 90,000 frames. Naïve patch tokenization is infeasible.

Practical strategies:

- **Aggressive temporal subsampling.** Sample 1 frame per second; tokenize each as an image. Loses fine motion but works for high-level understanding.
- **Spatiotemporal patches.** $16 \times 16 \times 4$ cubelets in space-time. Halves the frame-rate cost relative to per-frame patching.
- **Two-stream encoders.** Separate spatial and temporal encoders; fuse at the LLM. Used in older video-language work.
- **Token compression.** Pool, merge, or learn-to-summarize tokens within frame groups before feeding the LLM. Used by Video-LLaVA, VideoChat, and others.
- **Audio tokenization.** Encodec/SoundStream encode 24 kHz audio at ~ 75 Hz; an hour of audio is ~ 270 k tokens. Manageable but not cheap. Whisper-style continuous features are common as input; discrete tokens dominate for output (text-to-speech).

The architectural lesson: *time multiplies token counts, and token counts multiply attention's T^2* . Long-form video understanding is currently bandwidth-and-context-bounded by exactly the same wall that long-form text was, and the same fixes (sparse attention, SSMs, hierarchical token compression) apply. Multimodality does not change the architecture; it changes which constraints bind first.

Chapter 19. Reasoning & Test-Time Compute

19.1 Why current LLMs reason poorly inside the forward pass

A standard decoder-only LLM has a fixed compute budget per token: one forward pass through L layers. For tokens that require multi-step computation (math, planning, code synthesis), this budget is finite and often insufficient. The model has no native way to “think more” about a hard token.

Two architectural symptoms:

1. **Errors compound auto-regressively.** A wrong intermediate token derails the rest.
2. **No backtracking.** The model commits to each token; no internal mechanism revises an earlier output without external scaffolding.

19.2 Chain-of-thought as an external computational tape

Wei et al. (2022) and subsequent work showed that prompting models to “think step by step” lifts accuracy on math and reasoning benchmarks dramatically. The mechanism: each generated token gives the model another forward pass to do work in. Output tokens become a *scratchpad*. Effective compute scales linearly with output length.

This is, in a real sense, the LLM’s first form of *test-time compute*. The architecture didn’t change; the inference protocol did.

19.3 Verifier-based RL and inference scaling

The next step: train a separate verifier (often another LLM or a process reward model) that scores reasoning traces. At inference, sample many traces, score them, return the best (best-of-N) or run a more elaborate search (beam search, MCTS-like). At training time, RL finetuning (RLHF, RLAIF, DPO, GRPO) optimizes the policy to produce high-verifier-score traces.

This is what underlies the “reasoning” model class (OpenAI’s o-series, DeepSeek-R1, and similar). The core insight:

Inference compute is a substitute for parameter count, if you have a way to verify outputs.

For domains with cheap verifiers (math, code with unit tests), this works spectacularly. For domains with no verifier (open-ended writing, ethics), it works less well.

19.4 Inference scaling laws

Recent results (Snell et al. 2024; the o-series) suggest power-law scaling not only of training compute but of *inference* compute:

$$L_{\text{verified}} \approx \left(\frac{C_{\text{infer}}}{C_0} \right)^{-\gamma}.$$

The exponent γ depends on verifier quality and task. The implication: at fixed training cost, you can buy quality with inference. At fixed inference cost, you can buy quality with training. The optimal point depends on serving economics.

19.5 Why reasoning lives outside the forward pass

A forward pass is *width-limited* (parallel computation up to $\mathcal{O}(d)$) but *depth-limited* (only L layers). Iterative reasoning needs unbounded depth. The current solution — generate intermediate tokens, feed them back — is computationally a *recurrent* structure on top of a non-recurrent backbone. This is, mechanistically, a kludge.

What might come next, architecturally:

- **Latent reasoning loops.** Layers that iterate inside a single token’s forward pass (Universal Transformer, deep equilibrium models). Mostly empirically shaky so far.
- **Adaptive computation time.** Per-token decisions about how many layers to run (PonderNet, COCONUT). Promising but fragile.
- **Architectures that natively support search.** Trees, graphs, retrieval-as-thought. Largely speculative.

The lesson: **LLM reasoning is currently a system property, not an architectural property.** That gap is one of the largest open opportunities in the field.

19.6 Reward models: outcome vs process

Reinforcement learning on reasoning traces requires a reward signal. Two designs dominate.

- **Outcome Reward Models (ORM).** Score the final answer only. Cheap to train (binary correctness labels for many domains); cheap to apply at inference. Limitation: zero credit assignment within the trace. The model can stumble on bad reasoning that happens to produce a right answer (spurious success) or get the right reasoning steps and a wrong answer at the end (lost-Annie problem). Both teach the model the wrong lesson.
- **Process Reward Models (PRM).** Score each step in the reasoning trace. Provides denser gradient signal; reduces spurious-success problems. Cost: per-step labels are expensive — typically requiring expert annotation or careful synthetic generation — and the labels themselves can be noisy or wrong, especially in domains where “step correctness” is fuzzy.

Empirically, PRMs help substantially in domains where step-level correctness is well-defined (math, formal logic, multi-hop QA). ORMs are sufficient and cheaper in domains with verifier-friendly outcomes (executable code with unit tests, math with computable answers).

A hybrid pattern: train a PRM on a small expert-labeled set; train an ORM at scale on automated outcome verification; combine via reward shaping (e.g., final reward = $\alpha \cdot \text{ORM} + (1 - \alpha) \cdot \min_t \text{PRM}_t$, penalizing the worst step). DeepSeek-R1’s reported recipe leans heavily on outcome rewards from automated verifiers; OpenAI’s o-series is widely believed to use richer process supervision; both produce strong results, suggesting this design space has multiple workable points.

19.7 Search at inference

If a verifier is available, the policy can be a *prior* over reasoning traces and the verifier can drive a search. Approaches in roughly increasing engineering complexity:

- **Best-of-N.** Sample N independent traces, return the one with highest verifier score. Embarrassingly parallel; $\mathcal{O}(N)$ inference cost; quality scales sublinearly with N (often $\propto \log N$ at large N). The simplest baseline; surprisingly competitive.
- **Self-consistency** (Wang et al. 2022). Sample N traces; majority-vote on the answer. Cheaper than running a verifier; works when the answer space is small. Equivalent to best-of-N with a “popularity” verifier.
- **Beam search over reasoning steps.** Maintain a beam of partial traces, extend them step by step, score with a PRM, prune. Better quality per FLOP than best-of-N at large N because pruning concentrates compute on promising branches.
- **MCTS-flavored search.** Tree-structured exploration; balance exploitation (high-PRM children) and exploration (UCB-style bonuses). Heavier engineering; widely reported to power state-of-the-art reasoning models, though exact recipes are proprietary.
- **Lookahead-corrected decoding.** Speculatively decode several tokens, evaluate, accept or backtrack. Hybrid of generation and search. Active research direction.

The right comparison is to *training compute*. At fixed total compute (training + inference), the optimal allocation depends on the marginal return on each. Snell et al. (2024) report regimes where increasing inference compute outperforms increasing model size by 10× or more. This is exactly the constraint reallocation that the o-series template (\$19.8) operationalized.

19.8 The o-series template

Without claiming to know any proprietary recipe, the publicly described elements of OpenAI’s o-series and DeepSeek-R1 paint a consistent picture:

1. **A capable base LLM.** Standard pretraining; nothing exotic in the architecture. Usually a strong instruction-tuned model.
2. **Reasoning data.** Long traces with step-by-step structure (chain-of-thought, scratchpad, self-reflection, error-correction). Often synthetic, generated by the same model or a teacher and filtered by a verifier.
3. **An RL stage with a verifier.** GRPO, PPO, or DPO-flavored objectives; verifier (an ORM, PRM, or automated check) provides the reward signal; long roll-outs (thousands of tokens of reasoning) are common.
4. **Inference-time search or extended generation.** Best-of-N, beam search with a PRM, or simply long single-trace generation. The served model is the policy from step 3.

This template treats the *forward pass* as an unchanged Transformer and puts all reasoning capability into *training data + inference protocol*. It is, at the architectural level, a strong indictment of the dense Transformer’s reasoning capacity per forward pass — and an equally strong endorsement of the idea that *test-time compute*, properly steered, substitutes for parameter count.

The architectural question this leaves open: is the right next architecture one that bakes search and iterative refinement *into* the forward pass (latent reasoning, adaptive computation time, deep equilibrium models), or one that continues to externalize reasoning through tokens? The community is divided, the empirical evidence is incomplete, and the answer will likely be visible only in retrospect after another generation of models has been trained. What is clear is that the era in which “scale the dense Transformer” was a complete answer to capability questions is over.

Chapter 20. Comparative Architecture Analysis

This chapter consolidates everything in a few tables. Read these rows as the questions you should ask of any new proposed architecture.

20.1 Path length

The number of nonlinear steps a gradient must traverse from output position t to input position s .

Architecture	Path length
n -gram	0 (no learning across positions)
Bengio NPLM	1 (within window), ∞ outside
RNN	$ t - s $
LSTM/GRU	$ t - s $ along cell, but gated; effectively shorter
Bahdanau attention	1 (decoder→encoder), $ t - s $ within decoder
Transformer	1

Architecture	Path length
SSM (S4/Mamba)	Implicit; effective length depends on A 's spectrum

Short path lengths help credit assignment. This single column predicts a remarkable fraction of training stability.

20.2 Parallelism

Can the architecture compute its forward pass over a length- T sequence in parallel?

Architecture	Parallel along T ?
RNN/LSTM/GRU	No
Transformer (training)	Yes
Transformer (autoregressive inference)	No (but KV-cached)
SSM (training, conv form)	Yes
SSM (inference)	No, but $\mathcal{O}(1)$ per step
MoE	Yes, with all-to-all communication

Parallelism along T is the single property that separates “1990s LM technology” from “modern LM technology.”

20.3 Memory

Asymptotic memory cost as $T \rightarrow \infty$.

Resource	RNN	Transformer (train)	Transformer (infer)	SSM
Activations	$\mathcal{O}(T)$	$\mathcal{O}(T^2)$ naïve, $\mathcal{O}(T)$ with FlashAttention	—	$\mathcal{O}(T)$
State / KV cache	$\mathcal{O}(d)$	—	$\mathcal{O}(LTd)$	$\mathcal{O}(d)$

The state-vs-KV asymmetry is the entire reason long-context inference is hard with Transformers and easy with SSMs.

20.4 Compute

Training compute per layer for a length- T sequence.

Architecture	FLOPs per layer per sequence
RNN/LSTM	$\mathcal{O}(Td^2)$
Transformer	$\mathcal{O}(T^2d + Td^2)$
Sparse attention (window w)	$\mathcal{O}(Twd + Td^2)$
Linear attention	$\mathcal{O}(Td^2)$

Architecture	FLOPs per layer per sequence
SSM (conv)	$\mathcal{O}(T \log T \cdot d)$
SSM (scan)	$\mathcal{O}(Td)$
MoE (top- k over E experts)	Same FLOPs as dense / $k \cdot d^2 \cdot T$

Notice the recurring tradeoff between T scaling and d scaling. There is no architecture that wins both columns; choose your enemy.

20.5 Inductive bias

What does the architecture *prefer* to learn easily?

Architecture	Strong biases
n -gram	Local, count-based
Bengio NPLM	Smooth, similarity-based, fixed-length
RNN/LSTM	Sequential, decaying memory
Transformer	Content-addressed, no positional bias unless added
Convolutional LM	Local, translation-equivariant
SSM	Smooth time-invariant dynamics; selective with input dependence
MoE	Specialization-friendly
RAG	Retrieval-mediated factuality

The Transformer's *weak* inductive bias is sometimes a virtue (it doesn't waste capacity assuming the wrong thing) and sometimes a vice (it requires more data to discover what other architectures bake in for free). At scale, the virtue dominates.

20.6 Unified architecture summary

The five tables of this chapter can be collapsed into one summary at the cost of brevity:

Architecture	Train compute	Inference compute / token	KV/state mem	Path length	Hardware fit	Inductive bias	Where it wins
n -gram	$\mathcal{O}(D)$	$\mathcal{O}(1)$	$\mathcal{O}(V^n)$	n	CPU table-lookup	Local, count-based	Tiny, fast, interpretable
Bengio NPLM	$\mathcal{O}(D \cdot d^2)$	$\mathcal{O}(d^2)$	$\mathcal{O}(d)$	n	OK	Smooth, fixed-window	Historic; replaced
LSTM/GRU	$\mathcal{O}(Td^2)$	$\mathcal{O}(d^2)$	$\mathcal{O}(d)$	T	Poor (sequential)	Sequential, decay	Small-data regimes; mostly retired

Architecture	Train compute	Inference compute / token	KV/state mem	Path length	Hardware fit	Inductive bias	Where it wins
Dense Transformer	$\mathcal{O}(T^2d + Td^2)$	$\mathcal{O}(Td + d^2)$	$\mathcal{O}(Td)$	1	Excellent	Content-addressed	Almost everything at scale
MoE Transformer	Active-FLOP scaling	Active-FLOP scaling	$\mathcal{O}(Td)$ + experts	1	Good w/ all-to-all	Specialization	Capacity at fixed FLOPs
SSM (Mamba)	$\mathcal{O}(Td^2)$	$\mathcal{O}(d^2)$	$\mathcal{O}(Nd)$	T implicit	Good w/ scan	Smooth dynamics + selectivity	Long contexts, edge devices
Hybrid attn + SSM	Mostly $\mathcal{O}(Td^2)$	Mixed	Smaller than dense	Mixed	Good	Both	Long context with content-addressed reasoning
RAG-augmented	(LLM cost)	(LLM cost) + retrieval	(LLM) + index	1 in LLM	Good	Retrieval-mediated	Updateable knowledge, audit trails
LLM + verifier search	(LLM cost) $\times N$ samples	$\mathcal{O}(N)$ at inference	(LLM)	1 + tape	Good	Search-as-thought	Reasoning, code, math

This is the cheat sheet. Every architectural proposal can be plotted into this table; if its row does not strictly improve at least one column without making any other column materially worse, it is not progress — it is a relabeling.

20.7 Architectures by binding constraint

A complementary framing: *what is the binding constraint, and which architecture is best at each?*

Binding constraint	Best-of-class architecture	Why it wins
Training FLOPs (small data)	Dense Transformer, Chinchilla-scaled	Simplest, no auxiliary losses
Training FLOPs (capacity ceiling)	MoE Transformer	More parameters per FLOP
Inference latency / token	Dense Transformer with GQA + FlashAttention	KV-cache amortization, kernel-level optimization
Inference memory	SSM (Mamba), Hybrid	Fixed state, no T -scaling KV
Long-context quality	Dense Transformer with continued long-context training	Attention's content-addressed routing

Binding constraint	Best-of-class architecture	Why it wins
Long-context cost	SSM, sparse-attention Transformer, Hybrid	Avoids T^2
Updateable knowledge	RAG-augmented LLM	Edit the index, not the weights
Reasoning depth	LLM + verifier-driven inference search	Test-time compute substitutes for parameters
Modality coverage	Decoder-only LLM with multimodal tokenization	Single stack, multiple input/output formats
Edge / on-device inference	SSM or quantized small Transformer	Bounded state, low memory
Hardware-poor environments	Quantized dense Transformer	Mature kernels, predictable performance
Auditable / regulated deployments	RAG-augmented LLM with citations	Provenance is in the retrieval, not the weights

The thesis of this monograph in one table: *no architecture wins all binding constraints*. Mature engineering picks the binding constraint first and the architecture second. The future is a matrix of architectures, not a single winner — and the matrix’s cells are filled in by the binding-constraint analysis above.

Chapter 21. Open Problems & Tensions

21.1 Memory vs compute

The KV cache is the dominant inference cost for long contexts. Architectures that grow state with T (Transformers) trade favorable training compute for unfavorable inference memory. Architectures with fixed state (SSMs, RNNs) trade favorable inference for harder long-range training. Hybrids exist but no clear winner. The systems-level question — *can we afford to serve million-token contexts to billions of users?* — remains an open architectural problem in disguise.

21.2 Reasoning vs pattern matching

Are LLMs reasoning, or matching patterns from training data? The honest answer is: both, in proportions that vary by task and scale. Mechanistic interpretability finds genuine algorithms (induction, copying, modular arithmetic circuits) inside the residual stream. It also finds shortcuts and memorization. The architectural question is whether deeper or more iterative architectures can reduce the shortcut/algorithm ratio. So far, scale and inference-time compute help more than architectural changes.

21.3 Sparsity vs stability

MoE promises capacity without compute, but stability is brittle. Sparse activations (ReLU/Top-K) help interpretability but hurt smoothness; dense attention (softmax) is stable but expensive. Every “sparse” architecture (SwiGLU is partly sparse, MoE is sparse, mixture-of-depths is sparse, etc.) pays a stability tax.

We do not yet have a sparsity story that is *both* general and robust.

21.4 Generalization vs memorization

Scaling laws are compatible with both pure compression (memorization) and genuine abstraction (generalization). The two look identical from outside. As models approach the irreducible loss floor, the question of *what* they learned becomes harder to dodge: will the next 10× of compute give us a model that thinks better, or one that has memorized more? Architectural innovations that *bias toward generalization* (compositional structure, factored representations, explicit symmetry) remain underexplored.

21.5 Alignment as an architectural concern

RLHF/DPO/GRPO sit *on top of* a fixed base model. They reshape the policy without changing the architecture. Some open questions are arguably architectural: should there be a separate “values” module? A retrieval-augmented constitution? Explicit uncertainty estimates? These ideas are mostly speculative, but the more capable models become, the more architectural support for alignment may matter.

Chapter 22. How to Invent New Architectures

This is the operational chapter. If you are going to propose, build, or evaluate a new architecture, do it with a checklist.

22.1 The constraint table

Before writing any code, fill in a table for your proposal and at least two baselines:

Constraint	Your proposal	Transformer	Mamba
Training FLOPs / token			
Inference FLOPs / token			
Activation memory			
KV / state memory			
Path length (longest grad)			
Parallelism along T			
Hardware utilization (% of peak)			
Failure modes (list)			

If your proposal is not better in *at least one* row and *no worse* in the others, it is not an improvement; it is a relabeling.

22.2 Questions to ask

1. **What constraint am I relaxing?** Be specific. “Better” is not a constraint.
2. **What new constraint am I introducing?** There is always one. Find it.
3. **At what scale does my advantage appear?** Some innovations help at 100M and disappear at 10B. Some are reversed.
4. **What does my training compute curve look like?** Does it shift the prefactor? The exponent? Neither?
5. **What does my inference profile look like?** What is the bytes-moved per token, the KV size, the latency?
6. **What is my failure mode at extreme scale?** Routing collapse? Numerical instability? Bandwidth wall?
7. **Does my proposal rely on a kernel that exists?** If not, the systems work is half the project.
8. **Can my proposal be ablated cleanly?** If you can’t isolate the change, you can’t attribute the win.

22.3 Common failure patterns

- **Toy-scale wins that don’t survive scaling.** If you only run at 10M parameters, you have not measured anything real.
- **Compute-equivalent comparison violations.** Adding capacity is not innovation. Compare at iso-FLOPs, iso-parameters, iso-tokens.
- **Single-axis benchmarks.** A perplexity win is not a downstream win. Long-range Arena is not the world.
- **Memory-bandwidth blindness.** A FLOPs reduction with worse memory access pattern is often slower in practice.
- **Ignoring the residual stream.** Architectures that disrupt the additive residual structure have to overcome a stability tax that is usually invisible until late training.
- **Ignoring inference.** Most LLM compute is inference. Architectures that win training and lose inference will not be deployed.
- **No mechanism story.** “It worked” is not enough. If you can’t say *why*, the next group can’t iterate.
- **Reinventing recurrence accidentally.** Many “novel” architectures collapse to a known recurrence under analysis. Check.

22.4 What good research looks like

The strongest architecture papers of the last decade — Transformer, FlashAttention, Chinchilla, S4/Mamba, RoPE, GQA — share a pattern:

1. They identify a specific, measurable bottleneck.
2. They diagnose *why* it is a bottleneck (often via a careful complexity or hardware analysis).
3. They propose a minimal change that addresses it without breaking other constraints.
4. They demonstrate the change at multiple scales, with apples-to-apples baselines.
5. They explain *why* the change works, not just that it does.

Aspire to all five. Settle for nothing less than four.

Chapter 23. Conclusion

23.1 The arc, in one paragraph

We started with n -gram counting, hit a sparsity wall, replaced one-hot tokens with dense embeddings, hit a fixed-context wall, replaced fixed windows with recurrence, hit a gradient wall, replaced naïve recurrence with gating, hit a sequential-compute wall, replaced recurrence with attention, hit an $\mathcal{O}(T^2)$ wall, replaced dense attention with mixtures of sparse attention, retrieval, mixtures of experts, and state-space models, hit a reasoning wall, and started buying capability with inference-time compute. At each step the new architecture solved the binding constraint of its day and revealed the next.

23.2 What persists

A short list of lessons that survived multiple architectural revolutions:

- **Distributed representations beat exact-match counting.** Once. Always. Forever.
- **Additive paths are stable; multiplicative paths are not.** Residuals, gates, and CECs all rest on this.
- **Path length matters for credit assignment.** Shorter paths train faster and more reliably.
- **Hardware utilization is part of architecture.** A model that uses 10% of an accelerator's peak FLOPs is doing worse than its FLOP count suggests.
- **At sufficient scale, weak inductive biases are a feature.** Strong priors save data; weak priors save assumptions. Scale tilts the balance.
- **Compute is fungible across training and inference.** The optimal split changes; the total is what matters.

23.3 What is likely to come

Predicting architectures is mostly foolish, but a few tendencies are visible:

- **Hybrid stacks.** Dense attention layers interleaved with SSM or sparse-attention layers; dense plus MoE; reasoning modules wrapped around fast generators. The era of "one architecture all the way down" is probably ending.
- **Inference-time compute as a first-class citizen.** Architectures explicitly designed for variable-depth, verifier-driven inference rather than fixed forward passes.
- **Memory as a system, not a layer.** Retrieval, episodic stores, persistent KV summaries. The boundary between "model" and "system" continues to blur.
- **More physically motivated architectures.** Hardware (memory bandwidth, network topology, datacenter shape) will keep dictating which abstractions can be efficient. The architectures of 2030 will look the way they do because of what TSMC and NVIDIA shipped in 2027.

23.4 Final word

The goal of this monograph was not to teach the names of the models, but to install a habit of mind: *every architecture is a constraint reallocation; the relevant question about a new model is not "what is it?" but "what did it move?"*. If you carry that question with you into the next paper, the next benchmark, and the next training run, you will be doing the kind of analysis the field needs more of, and the kind of work the field's next breakthroughs will likely come from.

The history of LLM architectures is short — a few decades — but it has already taught us that progress is constraint-driven, that good ideas get rediscovered when hardware permits, and that the next breakthrough is usually visible in advance to those who keep careful constraint tables. Keep yours up to date.

Appendix A. Mathematical preliminaries

A self-contained refresher on the math used in this monograph. Skim if comfortable; return if confused.

A.1 Probability and information

A discrete random variable X over an alphabet $\mathcal{X}$ has a probability mass function $p(x)$ with $\sum_x p(x) = 1$. Its **entropy** is

$$H(p) = - \sum_x p(x) \log p(x),$$

measured in nats if $\log = \ln$ and bits if $\log = \log_2$. The entropy is the average information content of a sample drawn from p — and equivalently, the expected code length of an optimal prefix-free code for p .

The **cross-entropy** between distributions p (true) and q (model) is

$$H(p, q) = - \sum_x p(x) \log q(x).$$

Cross-entropy is the average code length when one uses a code optimal for q on samples actually drawn from p . Always $H(p, q) \geq H(p)$, with equality iff $p = q$.

The **Kullback–Leibler divergence** is the gap:

$$D_{KL}(p \parallel q) = H(p, q) - H(p) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

KL is non-negative, asymmetric ($D_{KL}(p \parallel q) \neq D_{KL}(q \parallel p)$ in general), and equals zero iff the two distributions agree.

LLM training minimizes cross-entropy, which — since the empirical entropy of the training corpus is fixed — is equivalent to minimizing the KL from the empirical distribution to the model. **Perplexity** is $\exp(H(p, q))$, the effective branching factor of the predictive distribution. Halving the loss multiplies the effective vocabulary the model can predict over.

A.2 Linear algebra

Vectors live in $\mathbb{R}^d$. Matrices act on them.

The **inner product** $\langle x, y \rangle = x^\top y$ measures alignment. Dot-product attention is built directly on this: similar query/key vectors yield large logits.

A matrix $W \in \mathbb{R}^{m \times n}$ has a singular value decomposition $W = U\Sigma V^\top$ with Σ diagonal, holding singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. The **operator norm** (or spectral norm) is $\sigma_1 = \|W\|_{\text{op}} = \sup_{\|x\|=1} \|Wx\|$.

For neural network analysis, two consequences matter:

- A linear layer's gradient norm scales with its operator norm.
- Stacking L linear layers produces a product whose operator norm is bounded by the *product* of the individual operator norms.

This is why depth without normalization or residual connections fails: small operator norms shrink gradients exponentially across depth; large operator norms blow them up. Residual connections add an identity to each layer's transformation, replacing the multiplicative product with something closer to a sum.

A.3 Calculus of softmax

The softmax $\pi : \mathbb{R}^d \rightarrow \Delta^{d-1}$ is

$$\pi_i(z) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Its Jacobian is

$$\frac{\partial \pi_i}{\partial z_j} = \pi_i(\delta_{ij} - \pi_j),$$

a matrix of the form $\text{diag}(\pi) - \pi\pi^\top$. This is symmetric, positive semi-definite, and rank-deficient (the all-ones vector is in its null space, reflecting softmax's invariance to additive constants).

Cross-entropy with softmax has a particularly clean gradient. If $\ell = -\log \pi_y(z)$ for target index y , then

$$\frac{\partial \ell}{\partial z_i} = \pi_i(z) - \delta_{iy} = \pi_i(z) - p_i,$$

where p is the one-hot target vector. The gradient is *the prediction error*: simple, bounded, and well-conditioned. There is no $1/\sigma$ blowup, no asymptotic vanishing. It is one of the few cases where the obvious choice turns out to be theoretically optimal.

A.4 Optimization

Stochastic gradient descent (SGD) updates $\theta \leftarrow \theta - \eta g$ with g a stochastic estimate of the gradient. Convergence requires η small relative to the largest curvature direction; the *condition number* (ratio of largest to smallest curvature) determines convergence rate.

Adaptive optimizers (Adam, AdamW, AdaFactor, Lion) use second-moment estimates to precondition the gradient, effectively rescaling each coordinate by its historical magnitude. This collapses the effective condition number and is the reason these optimizers converge on Transformer training where SGD with one global η often does not.

The price of adaptivity: optimizer state. Adam stores m and v for every parameter, doubling memory. Optimizer state often dominates training memory at scale, and reducing it (8-bit Adam, AdaFactor, ZeRO sharding) is a major practical concern.

A.5 Useful inequalities

- **Jensen.** For convex f : $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$. The basis of variational inequalities and the ELBO.
- **Cauchy–Schwarz.** $|\langle x, y \rangle| \leq \|x\| \|y\|$. The basis of cosine similarity.
- **Markov.** $\Pr[X \geq a] \leq \mathbb{E}[X]/a$ for non-negative X . The simplest concentration bound.
- **Chebyshev.** $\Pr[|X - \mu| \geq k\sigma] \leq 1/k^2$. Distribution-free concentration.
- **Hoeffding.** Sub-Gaussian variables concentrate exponentially. Why empirical estimates of expected losses converge fast.

These appear, often implicitly, in scaling-law derivations and convergence proofs throughout the literature this monograph references.

Appendix B. Annotated bibliography

A short, opinionated reading list. Papers in approximate order of when reading them is most useful for an aspiring architect.

Foundations

- *A Neural Probabilistic Language Model* — Bengio, Ducharme, Vincent, Jauvin (2003). The paper that introduced distributed word representations. Read it for the framing of sparsity as the central problem of statistical LMs.
- *Efficient Estimation of Word Representations in Vector Space* — Mikolov et al. (2013, “word2vec”). Famous for “king – man + woman = queen”; more importantly, established that distributional semantics is recoverable in low-dimensional vector spaces.

Recurrence

- *Long Short-Term Memory* — Hochreiter & Schmidhuber (1997). Required reading on the constant error carousel and the gating mechanisms that defined recurrent networks for two decades.
- *On the difficulty of training recurrent neural networks* — Pascanu, Mikolov, Bengio (2013). The clearest analysis of vanishing/exploding gradients in deep recurrence.

Attention and Transformers

- *Neural Machine Translation by Jointly Learning to Align and Translate* — Bahdanau, Cho, Bengio (2014). Attention is born here, in NMT. The motivation of escaping the encoder bottleneck remains the cleanest framing of why attention exists.
- *Attention Is All You Need* — Vaswani et al. (2017). The Transformer paper. Read it for the tables, the analysis of self-attention’s properties, and the elegance of the architecture.
- *Layer Normalization* — Ba, Kiros, Hinton (2016). Important predecessor; understanding why pre-norm beats post-norm requires understanding LayerNorm well first.

Pretraining and scaling

- *Improving Language Understanding by Generative Pre-Training* — Radford et al. (2018, "GPT"). The first GPT.
- *BERT: Pre-training of Deep Bidirectional Transformers* — Devlin et al. (2019). The MLM template.
- *Scaling Laws for Neural Language Models* — Kaplan et al. (2020). Foundational; even where superseded by Chinchilla, the methodology is the field standard.
- *Training Compute-Optimal Large Language Models* — Hoffmann et al. (2022, "Chinchilla"). Read carefully for the experimental design as much as the result.
- *Emergent Abilities of Large Language Models* — Wei et al. (2022). The original emergence paper.
- *Are Emergent Abilities of Large Language Models a Mirage?* — Schaeffer, Miranda, Koyejo (2023). The critique. Read both alongside each other.

Architectural choices

- *RoFormer: Enhanced Transformer with Rotary Position Embedding* — Su et al. (2021, "RoPE"). Now the default position encoding in nearly every open LLM.
- *Root Mean Square Layer Normalization* — Zhang & Sennrich (2019, "RMSNorm").
- *GLU Variants Improve Transformer* — Shazeer (2020). Where SwiGLU lives.
- *GQA: Training Generalized Multi-Query Transformer Models* — Ainslie et al. (2023). The standard inference-friendly attention.
- *Outrageously Large Neural Networks* — Shazeer et al. (2017). The Sparsely-Gated MoE original.
- *Switch Transformers* — Fedus, Zoph, Shazeer (2021). The MoE that scaled.

Long context and systems

- *FlashAttention* and *FlashAttention-2* — Dao et al. (2022, 2023). Essential for understanding the systems-level economics of attention. *FlashAttention-3* (Shah et al. 2024) extends to Hopper-class hardware.
- *Efficient Memory Management for Large Language Model Serving with PagedAttention* — Kwon et al. (2023, "vLLM").
- *Longformer, BigBird, Performer*. The sparse and linear attention canon.

State-space models

- *HiPPO: Recurrent Memory with Optimal Polynomial Projections* — Gu, Dao, Ermon, Ré (2020). The mathematical foundation.
- *Efficiently Modeling Long Sequences with Structured State Spaces* — Gu, Goel, Ré (2022, "S4").
- *Mamba: Linear-Time Sequence Modeling with Selective State Spaces* — Gu & Dao (2023).

Reasoning and test-time compute

- *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* — Wei et al. (2022). Where chain-of-thought comes from.
- *Let's Verify Step by Step* — Lightman et al. (2023). Process reward models.
- *Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters* — Snell et al. (2024). The case for inference scaling.

Mechanistic interpretability

- *In-context Learning and Induction Heads* — Olsson et al. (2022). The residual-stream view in action.
- *A Mathematical Framework for Transformer Circuits* — Elhage et al. (2021). Required for serious interpretability work.

Optimization and parameterization

- *Decoupled Weight Decay Regularization* — Loshchilov & Hutter (2019). Why AdamW.
- *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer* — Yang et al. (2022). The μ P transfer recipe.

This list is incomplete and biased. Any researcher who wants to invent something will benefit more from working through these papers carefully than from reading any review article (this one included).

Appendix C. Glossary

A short list of terms used throughout the book, ordered roughly by first appearance.

- **Token.** A unit of text after tokenization. May be a word, subword (BPE, WordPiece, Unigram), character, or byte.
- **Vocabulary.** The set of possible tokens; size V .
- **Embedding.** A dense vector representation of a token, position, or other discrete object.
- **Cross-entropy.** The standard loss for next-token prediction; equivalent to negative log-likelihood under the model.
- **Perplexity.** $\exp(\text{cross-entropy})$. The effective branching factor of the predictive distribution.
- **Context length (T).** Maximum number of tokens the model can attend to at once.
- **Model dimension (d or d_{model}).** Width of the residual stream and most internal representations.
- **Head dimension (d_k).** Per-head dimensionality in multi-head attention; typically $d_k = d/h$ when h heads share total budget d .
- **Residual stream.** The per-position vector that every Transformer layer reads from and writes to.
- **KV cache.** Stored keys and values from previous positions; allows incremental decoding without recomputing attention over the prefix.
- **Causal mask.** A lower-triangular mask preventing attention to future positions; turns a Transformer into a left-to-right autoregressive model.
- **Pre-norm / post-norm.** Whether normalization is applied inside the residual sublayer (pre-norm) or after the residual add (post-norm). Pre-norm is now standard.
- **RoPE.** Rotary positional embedding; encodes relative position via 2D rotations of query/key vectors.
- **ALiBi.** Linear attention bias; an alternative to RoPE with strong length extrapolation properties.
- **RMSNorm.** Root-mean-square normalization; a centering-free variant of LayerNorm.
- **SwiGLU.** A gated linear unit using the Swish activation; standard MLP for modern LLMs.
- **GQA / MQA.** Grouped-query / multi-query attention; KV-sharing strategies that reduce inference memory.
- **MoE.** Mixture-of-experts; sparse routing of tokens to expert subnetworks.
- **Expert parallelism.** Distributing experts across devices, with all-to-all communication moving tokens to their selected experts.
- **Token-choice / expert-choice routing.** Whether tokens pick experts (with capacity overflow risk) or experts pick tokens (with token-drop risk).
- **FlashAttention.** An I/O-aware attention implementation that tiles the computation to keep intermediate matrices in SRAM.
- **Online softmax.** Incremental softmax computation; the mathematical kernel that enables FlashAttention.

- **State-space model (SSM).** A linear (or selective) recurrence parameterized by matrices $\bar{A}, \bar{B}, \bar{C}, \bar{D}$.
 - **HiPPO.** A family of $\bar{A}$ matrices motivated by orthogonal-polynomial approximation of signal histories; provides the long-memory backbone of S4 and Mamba.
 - **Selective scan.** Mamba’s hardware-aware implementation of input-dependent SSMs via parallel associative scan.
 - **RAG.** Retrieval-augmented generation; conditioning generation on retrieved documents from an external corpus.
 - **Process Reward Model (PRM).** A model that scores intermediate steps in a reasoning trace.
 - **Outcome Reward Model (ORM).** A model that scores only the final output of a reasoning trace.
 - **Best-of-N.** Sampling N outputs and selecting the one with highest verifier score.
 - **Self-consistency.** Majority-vote across N sampled reasoning traces.
 - **Chinchilla scaling.** The compute-optimal allocation of parameters and tokens; for current Transformer training, roughly $D \approx 20N$.
 - **μ P.** Maximal update parameterization; a width-invariant parameterization that enables hyperparameter transfer across scales.
 - **AdamW.** Adam with decoupled weight decay; the production default optimizer for LLMs.
 - **Warmup.** A linear ramp of the learning rate from 0 to its peak value over the first 0.5–2% of training steps.
 - **PagedAttention.** Virtual-memory-style management of the KV cache; the basis of vLLM.
 - **YaRN, NTK-aware scaling, position interpolation.** Techniques for extending RoPE-trained models to longer contexts than they were trained on.
-

Appendix D. Worked numerical examples

This appendix grounds the previous chapters in concrete arithmetic. Each example takes a real-ish model and computes a real-ish quantity. Numbers are rounded for readability; specific deployments will differ within constant factors. The goal is not the precise number but the *shape* of the calculation.

D.1 Forward-pass FLOPs for LLaMA-3-70B

LLaMA-3-70B has approximately:

- $L = 80$ layers
- $d = 8192$ model dimension
- $h = 64$ attention heads with GQA, $h_{kv} = 8$ KV heads
- $d_k = 128$ per-head dimension
- $d_{ff} = 28,672$ feed-forward width (SwiGLU expansion)
- $V \approx 128,000$ vocabulary
- $N \approx 70 \times 10^9$ parameters total

Per-token forward FLOPs (all multiplies counted as one FLOP for accounting; multiply-add as two, following the standard $C \approx 2ND$ rule for forward and $C \approx 6ND$ for forward+backward):

- **Attention Q projection:** $d \cdot d = 8192^2 \approx 6.7 \times 10^7$ multiply-accumulates per token.
- **Attention K, V projections (GQA):** $2 \cdot d \cdot d \cdot (h_{kv}/h) = 2 \cdot 8192^2 \cdot (8/64) \approx 1.7 \times 10^7$.
- **Attention output projection:** $d^2 \approx 6.7 \times 10^7$.

- **Attention scores + value blend:** $\mathcal{O}(T \cdot d)$ per token, where T is current context length. At $T = 8,000$, this is $\sim 6.6 \times 10^7$ per token. At $T = 128,000$, it is $\sim 10^9$ per token — comparable to all the matmuls combined.
- **MLP up + gate (SwiGLU has two up-projections):** $2 \cdot d \cdot d_{\text{ff}} = 2 \cdot 8192 \cdot 28672 \approx 4.7 \times 10^8$.
- **MLP down:** $d_{\text{ff}} \cdot d \approx 2.3 \times 10^8$.

Per layer per token (excluding the position-dependent attention scoring): about 8×10^8 multiply-accumulates, or $\approx 1.6 \times 10^9$ raw FLOPs. Across 80 layers: $\sim 1.3 \times 10^{11}$ FLOPs per forward token. The standard estimate $C_{\text{fwd}} \approx 2N \approx 1.4 \times 10^{11}$ FLOPs per token agrees within rounding.

Training cost per token (forward + backward + weight update under bf16 mixed precision with master fp32) is $C \approx 6N \approx 4.2 \times 10^{11}$ FLOPs/token = 420 GFLOPs/token.

A single H100 SXM5 at full bf16 throughput delivers ~ 989 TFLOPS sustained. At 50% MFU (a healthy, achievable real-world number) it processes $\sim 5 \times 10^{14}$ FLOPs/s $\div 4.2 \times 10^{11}$ FLOPs/token $\approx 1,200$ training tokens/s/GPU. Training on Chinchilla-optimal $D \approx 1.4 \times 10^{12}$ tokens needs $\approx 1.2 \times 10^9$ GPU-seconds $\approx 333\text{k}$ GPU-hours. Real reports for LLaMA-3-70B exceed this because the model was deliberately over-trained beyond Chinchilla; the calculation scales linearly in D .

D.2 KV-cache for LLaMA-3-70B at 128k context

KV-cache memory is

$$\text{KV bytes} = 2 \cdot L \cdot T \cdot h_{\text{kv}} \cdot d_k \cdot \text{bytes/float}.$$

For LLaMA-3-70B with GQA ($h_{\text{kv}} = 8$), $T = 128,000$, fp16 (2 bytes):

$$2 \cdot 80 \cdot 128,000 \cdot 8 \cdot 128 \cdot 2 = 4.2 \times 10^{10} \text{ bytes} \approx 39 \text{ GB},$$

per request. A single H100 80GB has 80 GB of HBM, ~ 140 GB worth of model weights at fp16 (so the model is sharded across ≥ 2 H100s minimum). The KV cache for a single 128k-token request consumes most of the remaining memory on the second GPU. This is exactly why long-context serving needs KV quantization or paged attention — the math forces it.

Without GQA ($h_{\text{kv}} = h = 64$, as in vanilla MHA), the cache would be 8× larger: ~ 312 GB per request — completely infeasible on a single node. GQA's 8× memory reduction is what makes 128k context economically feasible at all on commodity hardware.

If we further quantize the KV cache to int4 (0.5 bytes/element) we get ~ 10 GB per 128k request, and a single H100 can serve several concurrent long-context users. KV quantization is therefore not an optimization — it is a precondition for production long-context serving.

D.3 Chinchilla-optimal training for a 10^{24} FLOP budget

Chinchilla scaling: $C \approx 6ND$, optimum at $D \approx 20N$. So $C \approx 120N^2$, giving $N \approx \sqrt{C/120}$.

For $C = 10^{24}$ FLOPs (a frontier-scale training run):

$$N \approx \sqrt{10^{24}/120} \approx 9.1 \times 10^{10} \approx 91\text{B parameters},$$

$$D \approx 20N \approx 1.8 \times 10^{12} \approx 1.8\text{T tokens.}$$

If you instead plan to inference-amortize aggressively, you might over-train: $D = 100N$, so $N \approx \sqrt{C/600} \approx 4.1 \times 10^{10}$, $D \approx 4.1 \times 10^{12}$. You get a 41B model trained on 4.1T tokens. Smaller, denser, and cheaper to serve at the cost of slightly higher training loss.

This is why LLaMA-3-8B exists despite Chinchilla saying you “should” train a $\sqrt{10^{23}/120} \approx 28\text{B}$ model on $\sim 580\text{B}$ tokens for the same compute budget: serving an 8B model at 15T tokens is a far better deal for an open-weights audience that will deploy on consumer hardware. Chinchilla minimizes training loss per training FLOP; serving economics minimize quality-per-dollar over the *deployment lifetime*.

D.4 FlashAttention savings for a 16k context

For a forward pass at $T = 16,384$, $d_k = 128$ head dim, batch $B = 1$, single head:

- Naïve attention materializes the $T \times T = 16384^2 \approx 2.7 \times 10^8$ -element score matrix in HBM. At fp16 (2 bytes), this is ~ 540 MB *per head, per layer*. For 64 heads $\times$ 80 layers, this is petabytes of HBM traffic across a forward pass. HBM bandwidth (~ 3 TB/s on H100) is the bottleneck, not arithmetic.
- FlashAttention tiles the computation so the $T \times T$ matrix never appears in HBM. HBM traffic is dominated by reading Q, K, V once: $3 \cdot T \cdot d_k \cdot 2 \approx 3 \cdot 16384 \cdot 128 \cdot 2 \approx 12.6$ MB per head per layer.

The bandwidth ratio is $\sim 40\times$ in favor of FlashAttention. The arithmetic count is roughly equal between naïve and FlashAttention; the wall-clock speedup follows the bandwidth ratio because attention is bandwidth-bound. In practice, FlashAttention-2 reports 2–4 $\times$ wall-clock speedup over previous best implementations and 10–20 $\times$ memory reduction for long contexts. FlashAttention-3 on Hopper uses async copies and warp specialization to push the speedup further.

The lesson: *the relevant cost is not always FLOPs; it is bytes moved*. Roofline analysis is mandatory.

D.5 MoE active parameter accounting

DeepSeek-V3 (671B total, 37B active per token):

- Total parameters: $N_{\text{total}} = 671\text{B}$
- Per-token active parameters: $N_{\text{active}} = 37\text{B}$
- Sparsity ratio: $37/671 \approx 5.5\%$

Per-token training FLOPs are governed by *active* parameters: $C \approx 6 \cdot 37 \times 10^9 \approx 2.2 \times 10^{11}$ FLOPs/token. Compare to a hypothetical dense 37B at the same per-token FLOPs and a dense 671B at $\sim 18\times$ higher per-token FLOPs.

The win is at fixed FLOPs: DeepSeek-V3 has 18 $\times$ the parameter capacity of a 37B dense at the same training cost per token. The cost is memory and communication: all 671B parameters must live somewhere (~ 1.3 TB at fp16, requiring tensor and expert parallelism across many GPUs), and the all-to-all communication must shuffle a fraction of the tokens to each expert per layer. For $E = 256$ experts, top- $k = 8$, capacity factor 1.0, every layer moves $T \cdot d \cdot k/E$ bytes around the cluster — modest per layer but multiplied by hundreds of layers per training step, becoming a non-trivial fraction of training time on anything but top-tier interconnects.

D.6 Cost of generating 1M tokens

For LLaMA-3-70B on an H100 cluster, single-batch single-stream generation:

- Per-token forward FLOPs: $\sim 1.4 \times 10^{11}$ (forward only, no backward).
- An H100 at 50% MFU does $\sim 5 \times 10^{14}$ FLOPs/s.
- Compute-bound throughput: $5 \times 10^{14} / 1.4 \times 10^{11} \approx 3,500$ tokens/s. *In practice*, decode is not FLOP-bound but memory-bound, and a single H100 produces closer to 30–60 tokens/s for a 70B model. The bottleneck is loading ~ 140 GB of weights from HBM into SRAM per generated token.
- 1M tokens at 50 tokens/s = 20,000 seconds ≈ 5.5 hours of single-GPU time.

Batching changes everything: with continuous batching (vLLM-style), 64 concurrent requests share the weight loads; tokens/s/GPU at the *cluster* level can exceed 1,000 with enough concurrent demand. Cost-per-token drops by an order of magnitude. This is why production serving optimizes for batch fill rate as obsessively as for raw model speed.

The architectural lesson: *single-token latency and aggregate throughput are different optimization problems*. Latency-bound serving (chatbots, code autocomplete) wants small models, fast kernels, short context, and speculative decoding. Throughput-bound serving (data labeling, batch summarization) wants large batches, KV-cache prefix sharing, and a tolerant latency budget. The same model can serve both regimes, but not optimally — and architectural choices that ignore this distinction (e.g., MoE routing decisions tuned only for batch-1 latency, or dense-attention pipelines for million-token batch jobs) leave large amounts of money on the table.

D.7 Scaling RoPE: how far can you stretch?

RoPE rotates 2D query/key pairs at $d_k/2$ different frequencies $\theta_i = b^{-2i/d_k}$ for base $b = 10,000$ (LLaMA default). The lowest frequency $\theta_{d_k/2-1}$ corresponds to a wavelength of approximately $b = 10,000$ tokens. Beyond that wavelength, the rotation has cycled through an integer number of periods and the position information aliases.

Position interpolation rescales the position index by $T_{\text{train}}/T_{\text{eval}}$, which is equivalent to dividing all θ_i by the same factor. This stretches the wavelengths uniformly; the model now sees positions within a wavelength of $b \cdot T_{\text{eval}}/T_{\text{train}}$, but it has never been trained on the resulting frequency mix. Quality degrades smoothly up to maybe 2–4× extension before degradation becomes severe.

YaRN scales high-frequency components mildly (preserving short-range fidelity) and low-frequency components aggressively (extending long-range reach), with a smooth interpolation between regimes. Empirically YaRN handles 8–16× extensions cleanly; further extension still requires continued pretraining at the new context length to fully realize quality.

The lesson: *positional encodings are not just bookkeeping*. They define what “long context” means to the model. RoPE’s frequency spectrum is a hidden architectural parameter; tuning it well is what separates “supports 128k tokens” from “actually uses information at 128k tokens.”

Appendix E. Canonical Python Implementations

These snippets are *illustrative*, not production. They use NumPy/PyTorch idioms for clarity over speed. Each is annotated with the architectural ideas it embodies. Together they implement, in roughly 200 lines of code, the entire core of a modern Transformer LLM — plus illustrative versions of the key alternative kernels (FlashAttention, MoE, SSM, KV cache).

E.1 Self-attention from scratch

```
import torch
import torch.nn.functional as F

def self_attention(X, W_Q, W_K, W_V, mask=None):
    """
    X: (B, T, d) input sequence
    W_Q, W_K, W_V: (d, d) projection matrices
    mask: (T, T) additive mask with 0 for allowed, -inf for blocked
    Returns: (B, T, d)
    """
    Q = X @ W_Q                                # (B, T, d)
    K = X @ W_K
    V = X @ W_V
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / d_k**0.5  # (B, T, T)
    if mask is not None:
        scores = scores + mask
    attn = F.softmax(scores, dim=-1)             # (B, T, T)
    return attn @ V                             # (B, T, d)
```

The four lines that matter: project to QKV, score by dot product, normalize, weight values. Everything else in modern attention is optimization on top of this skeleton.

E.2 Multi-head attention

```
class MultiHeadAttention(torch.nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        assert d % n_heads == 0
        self.n_heads = n_heads
        self.d_k = d // n_heads
        self.W_Q = torch.nn.Linear(d, d, bias=False)
        self.W_K = torch.nn.Linear(d, d, bias=False)
        self.W_V = torch.nn.Linear(d, d, bias=False)
        self.W_O = torch.nn.Linear(d, d, bias=False)

    def forward(self, X, mask=None):
        B, T, d = X.shape
        # Project then reshape to (B, n_heads, T, d_k)
        def split(M):
```



```

        return M.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
    Q, K, V = split(self.W_Q(X)), split(self.W_K(X)), split(self.W_V(X))
    scores = Q @ K.transpose(-2, -1) / self.d_k**0.5
    if mask is not None:
        scores = scores + mask
    attn = scores.softmax(-1)
    out = (attn @ V).transpose(1, 2).contiguous().view(B, T, d)
    return self.W_O(out)

```

The reshape-transpose dance is what makes “multiple heads in parallel” a single batched matmul rather than a Python-level loop. GQA (grouped-query attention) modifies this to share K and V across groups of heads — a small change to the `split` helper that yields large KV-cache savings at inference time.

E.3 Causal mask

```

def causal_mask(T, device='cpu'):
    """Returns a (1, 1, T, T) tensor; -inf above diagonal, 0 on/below."""
    m = torch.full((T, T), float('-inf'), device=device)
    return torch.triu(m, diagonal=1).view(1, 1, T, T)

```

The mask is added *before* softmax; tokens at $-\infty$ contribute zero after exponentiation. This is how decoder-only LLMs enforce left-to-right attention with no architectural change to the attention kernel itself.

E.4 RoPE: Rotary Position Embeddings

```

def precompute_rope(d_k, max_T, base=10000.0, device='cpu'):
    """
    Returns cos and sin tensors of shape (max_T, d_k) for use in rotation.
    """
    inv_freq = 1.0 / (base ** (torch.arange(0, d_k, 2, device=device).float() / d_k))
    t = torch.arange(max_T, device=device).float()
    freqs = torch.outer(t, inv_freq)  # (max_T, d_k/2)
    cos = freqs.cos().repeat_interleave(2, dim=-1)  # (max_T, d_k)
    sin = freqs.sin().repeat_interleave(2, dim=-1)
    return cos, sin

def apply_rope(x, cos, sin):
    """
    x: (... , T, d_k). Rotates pairs (x_{2i}, x_{2i+1}) by precomputed angles.
    """
    T = x.size(-2)
    x1 = x[ ..., 0::2]
    x2 = x[ ..., 1::2]
    rotated = torch.stack([-x2, x1], dim=-1).flatten(-2)
    return x * cos[:T] + rotated * sin[:T]

```


The trick: apply the rotation to Q and K *before* the dot product. The relative position $t - s$ then falls out of the inner-product structure with no additional learned parameters. RoPE generalizes from absolute to relative position essentially for free.

E.5 RMSNorm

```
class RMSNorm(torch.nn.Module):
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.weight = torch.nn.Parameter(torch.ones(d))
        self.eps = eps

    def forward(self, x):
        rms = x.pow(2).mean(-1, keepdim=True).add(self.eps).sqrt()
        return self.weight * (x / rms)
```

Compare with LayerNorm: no mean subtraction, no bias parameter. Cheaper, equally effective in practice, now standard from LLaMA onward.

E.6 SwiGLU MLP

```
class SwiGLU(torch.nn.Module):
    def __init__(self, d, d_ff):
        super().__init__()
        self.W_up = torch.nn.Linear(d, d_ff, bias=False)
        self.W_gate = torch.nn.Linear(d, d_ff, bias=False)
        self.W_down = torch.nn.Linear(d_ff, d, bias=False)

    def forward(self, x):
        return self.W_down(F.silu(self.W_gate(x)) * self.W_up(x))
```

Two parallel linear projections; one is gated by a Swish nonlinearity ($\text{SiLU}(x) = x\sigma(x)$); their elementwise product passes through the down-projection. Replaces the classical `linear` $\rightarrow$ `ReLU` $\rightarrow$ `linear` MLP. The gating gives the network a multiplicative interaction the additive MLP lacks.

E.7 Online softmax (FlashAttention kernel idea)

```
def online_softmax_attention(Q, K, V, block_size=64):
    """
    Tiles K and V; computes softmax incrementally without materializing
    the full T x T score matrix. Single-head pedagogical version.
    Demonstrates the math of FlashAttention; not optimized.
    """
    T, d_k = Q.shape
    d_v = V.size(-1)
    O = torch.zeros(T, d_v)           # running output
    m = torch.full((T,), float('-inf')) # running max
    l = torch.zeros(T)                # running denominator
```

```

for j in range(0, T, block_size):
    K_block = K[j:j + block_size]
    V_block = V[j:j + block_size]
    S = Q @ K_block.T / d_k**0.5          # (T, B)
    m_block = S.max(-1).values            # (T,)
    m_new = torch.maximum(m, m_block)
    # Rescale running quantities to the new max for numerical stability.
    alpha = (m - m_new).exp().unsqueeze(-1)
    beta = (S - m_new.unsqueeze(-1)).exp() # (T, B)
    l = (alpha.squeeze(-1) * l) + beta.sum(-1)
    O = alpha * O + beta @ V_block
    m = m_new
return O / l.unsqueeze(-1)

```

The key idea is the recurrence on (m, l, O) : each new tile updates these without materializing the full $T \times T$ score matrix. Production FlashAttention adds tiling on Q as well, register/SRAM allocation, and asynchronous data movement — but the math is exactly this. The HBM traffic drops from $\mathcal{O}(T^2)$ to $\mathcal{O}(T \cdot d_k)$, which is the source of the 2–4× wall-clock speedup.

E.8 Top- k MoE router

```

class TopKMoE(torch.nn.Module):
    def __init__(self, d, n_experts, k=2):
        super().__init__()
        self.k = k
        self.gate = torch.nn.Linear(d, n_experts, bias=False)
        self.experts = torch.nn.ModuleList([
            SwiGLU(d, 4 * d) for _ in range(n_experts)
        ])

    def forward(self, x):
        # x: (B, T, d). Flatten batch and time for routing.
        B, T, d = x.shape
        x_flat = x.view(-1, d) # (N, d)
        logits = self.gate(x_flat) # (N, E)
        weights, idx = logits.topk(self.k, dim=-1)
        weights = weights.softmax(-1) # normalize across chosen experts
        out = torch.zeros_like(x_flat)
        for e, expert in enumerate(self.experts):
            mask = (idx == e).any(-1) # tokens that picked expert e
            if not mask.any():
                continue
            token_inputs = x_flat[mask]
            expert_out = expert(token_inputs)
            # Weight by the gating coefficient corresponding to expert e.
            w = (weights * (idx == e).float()).sum(-1, keepdim=True)
            out[mask] += w[mask] * expert_out
        return out.view(B, T, d)

    def load_balancing_loss(gate_logits, idx, n_experts):

```

```

"""Switch Transformer auxiliary loss: encourages uniform expert use."""
probs = gate_logits.softmax(-1).mean(0)          # (E,) avg routing prob
one_hot = F.one_hot(idx, n_experts).float()
fraction = one_hot.mean(0).mean(-1) if one_hot.dim() > 2 else one_hot.mean(0)
return n_experts * (probs * fraction).sum()

```

Production routers add capacity factors, expert-parallel collective comms, and avoid the Python-level loop over experts via grouped GEMMs. This implementation is correct but slow; it is meant to make the routing logic visible. The auxiliary loss is essential — without it, the gate collapses onto a few experts and the rest die.

E.9 Mamba-style associative scan (illustrative)

```

def selective_scan(u, A, B, C):
    """
    u: (T, d_in)          input sequence
    A: (T, d_state, d_state) per-step state transition (often diagonal)
    B: (T, d_state, d_in)
    C: (T, d_out, d_state)
    Returns y: (T, d_out)
    Sequential reference; production version uses a parallel scan in O(log T) depth.
    """
    T = u.size(0)
    d_state = A.size(1)
    h = torch.zeros(d_state)
    ys = []
    for t in range(T):
        h = A[t] @ h + B[t] @ u[t]
        y = C[t] @ h
        ys.append(y)
    return torch.stack(ys)

```

A real Mamba kernel computes this via a parallel associative scan (Blelloch’s algorithm) and runs the entire sequence in $\mathcal{O}(\log T)$ depth on a GPU. The math above is what the kernel computes. The “selective” in selective scan refers to making A_t, B_t, C_t functions of u_t — the input modulates the dynamics, giving content-dependent recurrence.

E.10 KV cache with sliding window and attention sinks

```

class SlidingKVCache:
    """
    Keeps the most recent W tokens plus the first S 'sink' tokens.
    Sink tokens (Xiao et al. 2023, StreamingLLM) are kept indefinitely
    because the attention softmax has unavoidable mass at the sequence
    start; evicting them destabilizes generation.
    """
    def __init__(self, n_layers, n_heads, d_head, window=4096, sinks=4, device='cpu'):
        self.W, self.S = window, sinks
        self.cache = [

```

```

        {'K': torch.zeros(n_heads, 0, d_head, device=device),
         'V': torch.zeros(n_heads, 0, d_head, device=device)}
        for _ in range(n_layers)
    ]

    def append(self, layer, k_new, v_new):
        c = self.cache[layer]
        K = torch.cat([c['K'], k_new], dim=1)
        V = torch.cat([c['V'], v_new], dim=1)
        # Keep first S (sinks) and last W (window) tokens.
        if K.size(1) > self.S + self.W:
            K = torch.cat([K[:, :self.S], K[:, -self.W:]], dim=1)
            V = torch.cat([V[:, :self.S], V[:, -self.W:]], dim=1)
        c['K'], c['V'] = K, V
        return c['K'], c['V']

```

The naive approach — keep the most recent W tokens, throw away the rest — fails empirically because the softmax always concentrates a non-trivial fraction of its mass at the start of the sequence (the “attention sink” phenomenon). Keeping $S = 4$ sink tokens stabilizes generation indefinitely while keeping the cache bounded.

E.11 Cosine LR schedule with warmup

```

import math

def cosine_warmup_lr(step, max_lr, min_lr, warmup_steps, total_steps):
    if step < warmup_steps:
        return max_lr * step / warmup_steps
    if step > total_steps:
        return min_lr
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return min_lr + 0.5 * (max_lr - min_lr) * (1.0 + math.cos(math.pi * progress))

```

The standard schedule for LLM pretraining: linear warmup for ~1% of steps (avoids the early-training spike that wrecks Adam’s moment estimates), cosine decay to ~10% of peak LR over the remaining budget. Departures from this are rare and almost always harmful.

E.12 Compute-optimal scaling-law solver

```

def chinchilla_optimal(C, a=406.4, b=410.7, e=1.69, alpha=0.34, beta=0.28):
    """
    Given a compute budget C (in FLOPs), return the compute-optimal (N, D)
    where N = parameters and D = training tokens. Coefficients are from
    Hoffmann et al. (2022) Chinchilla fit to L(N, D).
    The compute constraint  $C \sim 6 N D$  dictates the trade-off; the optimal
    point sets  $dL/dN$  scaled by D equal to  $dL/dD$  scaled by N.
    """
    # Closed form from the Chinchilla parametric loss:
    #  $N_{\text{opt}}(C) = G * (C/6)^a$ ,  $D_{\text{opt}}(C) = G^{-1} * (C/6)^b$ 

```

```

# where (a, b) sum to 1 and G depends on the curve fit.
G = (alpha * a / (beta * b)) ** (1 / (alpha + beta))
a_exp = beta / (alpha + beta)
b_exp = alpha / (alpha + beta)
N_opt = G * (C / 6) ** a_exp
D_opt = (1 / G) * (C / 6) ** b_exp
return N_opt, D_opt

# Example: a 1e24 FLOP budget
N, D = chinchilla_optimal(1e24)
print(f"Optimal: {N/1e9:.1f}B params, {D/1e9:.1f}B tokens, ratio D/N = {D/N:.1f}")

```

The 1:20 parameters-to-tokens ratio is the famous Chinchilla rule: at compute optimum, train roughly 20 tokens per parameter. Modern open-source models train at 100–300 tokens per parameter — they are *inference-optimal*, not compute-optimal, accepting more training compute to reduce serving cost.

E.13 A minimal Transformer block (assembly)

```

class TransformerBlock(torch.nn.Module):
    def __init__(self, d, n_heads, d_ff):
        super().__init__()
        self.norm1 = RMSNorm(d)
        self.attn = MultiHeadAttention(d, n_heads)
        self.norm2 = RMSNorm(d)
        self.mlp = SwiGLU(d, d_ff)

    def forward(self, x, mask=None):
        x = x + self.attn(self.norm1(x), mask=mask)
        x = x + self.mlp(self.norm2(x))
        return x

class TinyLM(torch.nn.Module):
    def __init__(self, vocab_size, d=256, n_heads=4, n_layers=4, d_ff=1024):
        super().__init__()
        self.tok_emb = torch.nn.Embedding(vocab_size, d)
        self.blocks = torch.nn.ModuleList([
            TransformerBlock(d, n_heads, d_ff) for _ in range(n_layers)
        ])
        self.norm = RMSNorm(d)
        self.head = torch.nn.Linear(d, vocab_size, bias=False)

    def forward(self, idx):
        B, T = idx.shape
        mask = causal_mask(T, device=idx.device)
        x = self.tok_emb(idx)
        for block in self.blocks:
            x = block(x, mask=mask)
        x = self.norm(x)
        return self.head(x)

```

This is the entire architecture of a modern LLM, in roughly sixty lines of PyTorch. Everything else — RoPE instead of absolute positions, GQA instead of MHA, MoE in place of the MLP, gradient checkpointing, distributed training, FlashAttention kernels, KV caching for inference — is engineering on top of this skeleton.

The architectural lesson: the *idea* of a Transformer is small. The *system* of a Transformer is enormous. Confusing the two is one of the most common research mistakes: papers that claim a new architecture sometimes turn out to be presenting a new *system* (better optimizer, better data, better tokenizer) wrapped around an only-slightly-modified Transformer. The discipline of separating “architecture” from “system” is a precondition for inventing new architectures rather than rebranding old ones.

End of monograph.